

Authenticated Garbling with Tensor Gates

David Heath¹, Nakul Khambhati², Rafail Ostrovsky², and Turan Vural²

¹University of Illinois Urbana-Champaign, daheath@illinois.edu

²UCLA, nakul@cs.ucla.edu, rafail@cs.ucla.edu, turan@cs.ucla.edu

Abstract

Authenticated garbling, introduced by Wang et al., CCS'17, is a leading paradigm for achieving constant-round maliciously-secure 2PC. In this work, we upgrade authenticated garbling to efficiently support tensor gates (introduced in the context of semi-honest garbling by Heath et al., CCS'21) using the *one-hot garbling* technique. Our maliciously-secure garbled tensor gate computes $\mathbf{x} \otimes \mathbf{y}$ for $\mathbf{x} \in \{0, 1\}^n, \mathbf{y} \in \{0, 1\}^m$ with $O(n + m)\kappa + O(nm)$ bits of communication, where κ is the computational security parameter and n, m are logarithmic in κ . This improves the best prior constant-round maliciously secure approach, which incurs $O(nm)\kappa$ communication. Our protocol is concretely efficient and allows improvement over state-of-the-art for applications including integer multiplication, matrix multiplication, and more. We benchmark the online phase of our protocol and observe a $5.41\times$ improvement in communication and $3.35\times$ improvement in wall clock time when computing a 128×128 bit tensor product.

Contents

1	Introduction	1
1.1	Our Contribution	2
2	Preliminaries	2
2.1	Notation	2
2.2	Circuits with Tensor Gates	3
2.3	Hash Functions	4
2.4	Information-Theoretic Message Authentication Codes	4
2.5	(m, L) -Independent Matrix	6
2.6	One Hot Garbling	7
3	Overview	7
3.1	Distributed Garbling with Tensor Gates	8
3.2	Consistency Check via Dual Execution	11
4	Distributed Garbling with Tensor Gates	13
4.1	Distributed Garbling	13
4.2	Distributed Garbling with Tensor Gates	15
5	Authenticated Garbling with Tensor Gates	20
A	Proof of Theorem 1	25
B	Applications	28
B.1	Generalized Outer Product	28
B.2	Binary Matrix Multiplication	28
B.3	Integer Multiplication	28
C	Experiments	29
C.1	Methodology	29
C.2	Results	29
D	Equivalence of Distributed Half-Gates	31
E	Compressed Preprocessing [CWYY23]	33
E.1	Communication Complexity	33

1 Introduction

Yao’s Garbled Circuit (GC) is a cryptographic technique that allows a *garbler* G to construct from some program P a *privacy-preserving* encoding of that program $\tilde{P}$. G sends this encoding to an *evaluator* E —along with an encoding of program input $\tilde{x}$ —who can locally compute $\tilde{P}(\tilde{x}) = P(x)$ *while keeping x hidden*. GC can be understood as a *randomized encoding* of computation [App17]. GC is particularly relevant to the study of secure two-party and multi-party computation (2PC and MPC), where it is used to achieve protocols that complete within only a small constant number of communication rounds. GC can be combined with OT to achieve semi-honest-secure 2PC protocols in a relatively straightforward manner; see e.g. [LP04]. The state-of-the-art concretely efficient semi-honest GC protocol for boolean circuits [RR21] communicates $\approx 1.5\kappa$ bits per AND gate, where κ is the computational security parameter, and XOR gates are free.

Maliciously-Secure Garbling Protocols Many garbling techniques are insecure against a malicious G . Namely, G can incorrectly garble a circuit such that an honest E obtains an incorrect output. This insecurity is somewhat inherent to many garbling techniques, as often the very mechanisms that protect G ’s privacy also prevent E from detecting that the circuit has been incorrectly garbled.

The cut-and-choose technique [LP15, KNR⁺17] can be applied to Yao’s GC (and its variants) in a black-box way to achieve malicious security. This involves G sending or committing ρ many GCs to E , where ρ is the statistical security parameter. E then randomly selects a subset for G to “open”. E checks these were properly constructed, and gains confidence in the unopened subset. The parties use the unopened GCs for secure evaluation. Cut-and-choose garbling incurs a large multiplicative factor overhead in cost.

The authenticated garbling framework [WRK17], which is the state of the art for maliciously secure GC protocols, adopts a different approach: G sends only one, potentially incorrect GC, which E evaluates and checks correctness of obtained values only *post evaluation*. For security, it is required that both G and E feed randomness into the GC and E holds commitments to G ’s randomness. The overhead this incurs can be moved entirely to a function-independent preprocessing phase, which leads to an online phase that roughly matches state of the art semi-honest protocols. Recent protocols [DILO22, CWYY23] have discovered communication-efficient methods to generate the required preprocessing, leading to an actively secure GC protocol that has almost the same total communication as the state of the art semi-honest GC protocol for boolean circuits.

Garbling Beyond Boolean Circuits: In recent years, the semi-honest GC literature has evolved to include concretely-efficient ways to garble *non-boolean-circuit structures*. In this discussion, we only consider GC schemes that rely on efficient symmetric-key cryptography. For instance, the literature includes special-purpose techniques for garbling RAM programs [LO13, HKO22, PLS23, HKO23, GTRS25], programs with conditional branching [HK20, HK21b], and arithmetic circuits [BMR16].

While these semi-honest techniques can be upgraded to the malicious security setting via cut and choose, the state-of-the-art authenticated garbling technique has not been able to keep up with the semi-honest progress. Indeed, the only example of authenticated garbling for non-boolean-circuit-based computation is the authenticated Garbled RAM of [HKO23], which uses a WRK-style [WRK17] approach to authenticate wires in so-called tri-state circuits. In particular, this technique still involves garbling of individual gates, each with inputs of length ≤ 2 .

One particularly exciting semi-honest GC technique is the so-called *one-hot garbling* technique [HK21a]. This technique reduces the communication cost of garbling certain *tensor operations*, such as matrix multiplication, and it offers concretely efficient solutions for certain $O(\log \kappa)$ -input gates. The one-hot garbling technique can be understood as a *generalization* of the fan-in-two “half gates” approach of [ZRE15], a technique that remains state of the art in the authenticated garbling setting [KRRW18]. The low-level nature of one hot garbling has made it useful as a building block for other semi-honest techniques, including garbled lookup tables [HKN24, KN25] (generalizations from fan-in-two gates to arbitrary n -row, m -output lookup tables) and garbled arithmetic circuits [Hea24].

Until now, it was not known how to reconcile one-hot-style techniques with authenticated garbling. This technique is noteworthy because it provides a communication versus computation tradeoff which offers a

concrete speed up in runtime in the symmetric-key based GC setting where network bandwidth is usually the bottleneck. This motivates our central research question:

Can one-hot garbling [HK21a] be used to obtain efficient authenticated garbling protocols for tensor products, lookup tables, arithmetic operations, etc?

1.1 Our Contribution

We take a major step towards answering the above in the affirmative by constructing an authenticated GC (constant-round maliciously secure 2PC) that handles tensor gates efficiently, making black-box use of the one-hot garbling technique. A tensor gate computes $\mathbf{x} \otimes \mathbf{y}$ for inputs $\mathbf{x} \in \{0,1\}^n, \mathbf{y} \in \{0,1\}^m$ within an authenticated garbled circuit. Our protocol garbles a tensor gate with total communication $O(n+m)\kappa + O(mn)$ bits with small hidden constants, provided n, m are at most logarithmic in the security parameter κ . In more detail, our protocol can garble a tensor gate with input lengths $n, m = c \log \kappa$ with $O(n+m)\kappa$ bits of communication and $O(\kappa^c \log \kappa)$ hash invocations, for any $c > 0$. Our construction makes black-box use of *semi-honest* privacy-free tensor gates [HK21b] to build an *authenticated* privacy-preserving tensor gate. This unifies and extends the techniques of [KRRW18] that make black-box use of semi-honest privacy-free AND gates [ZRE15] to build an authenticated privacy-preserving AND gate. To explain the novel insights underlying our construction, we first reformulate known GC protocols [ZRE15, KRRW18, HK21b] in a way that naturally leads to our proposed solution.

Our construction achieves an asymptotic and concrete reduction in the size of a garbled material for a tensor gate from $O(nm)\kappa$ to $O(n+m)\kappa$ bits. To achieve our desired asymptotic reduction in *total* communication, we adopt the dual execution technique which is compatible with compressed preprocessing so that the per-gate communication in the preprocessing phase is $O(nm)$ bits, independent of κ . Our final protocol $\Pi_{2\text{pc-tensor}}$ achieves maliciously secure 2PC while handling tensor gates efficiently.

Finally, we present concrete applications of our protocol, building on the work from the semi-honest setting [HK21b]. In particular, we first show how “small” tensor gates can be tiled to handle arbitrary length tensor products efficiently. This technique enables us to garble a tensor gate with input lengths n, m with $O(\frac{nm}{\log \kappa})$ bits of communication and $\text{poly}(\kappa)$ hash computations. We provide exact costs in Appendix B and benchmark a Rust implementation of our online phase in Appendix C, which shows a $5.41\times$ improvement in communication and $3.35\times$ improvement in wall clock time when computing a 128×128 bit tensor product. We describe how this speeds up integer and matrix multiplication within an authenticated GC. Finally, we remark that secure matrix multiplication is a key component of many AI applications, including neural network implementations [KNR⁺17, WGC19, WTB⁺21].

2 Preliminaries

2.1 Notation

We denote the computational and statistical security parameters using κ and ρ , respectively. We write $\log$ to denote base-2 logarithms. Given a finite set S , we write $x \leftarrow S$ to denote sampling x uniformly at random from S .

We use bold symbols (e.g., $\mathbf{x}, \mathbf{X}$) to denote vectors and matrices. We write x_i for the i -th component of vector $\mathbf{x}$, and $X_{i,j}$ for the (i,j) -th entry of matrix $\mathbf{X}$. Given an index set S , the notation $\{x_i\}_{i \in S}$ denotes the collection of values indexed by S .

Many works [BMR90, WRK17, KRRW18, CWYY23] make use of a public selection bit embedded in wire labels, typically as the least significant bit. We denote the extraction of this bit by $\text{ExtBit}(\cdot)$, following the notation of [CWYY23].

We write $\mathbb{F}_{2^\kappa}$ for an extension field of $\mathbb{F}_2$, which we instantiate using an irreducible polynomial $f(X)$ of degree κ . Any element $x \in \mathbb{F}_{2^\kappa}$ is uniquely expressed as a polynomial $\sum_{i=0}^{\kappa-1} x_i X^i$ with coefficients $x_i \in \mathbb{F}_2$. This induces an isomorphism between $\mathbb{F}_{2^\kappa}$ and $\mathbb{F}_2^\kappa$, allowing us to use $\{0,1\}^\kappa, \mathbb{F}_2^\kappa$, and $\mathbb{F}_{2^\kappa}$ interchangeably. In particular, addition in $\mathbb{F}_2^\kappa$ and $\mathbb{F}_{2^\kappa}$ corresponds to bitwise XOR in $\{0,1\}^\kappa$.

We define the following Bit-to-Field and Field-to-Bit conversion macros:

- **B2F**(x): Converts $x = (x_0, \dots, x_{\kappa-1}) \in \mathbb{F}_2^\kappa$ to $x = \sum_{i=0}^{\kappa-1} x_i X^i \in \mathbb{F}_{2^\kappa}$.
- **F2B**(x): Converts $x = \sum_{i=0}^{\kappa-1} x_i X^i \in \mathbb{F}_{2^\kappa}$ to $x = (x_0, \dots, x_{\kappa-1}) \in \mathbb{F}_2^\kappa$.

We denote garbling and evaluation using **Garble**, **Eval**, and refer to **G**, **E** as the garbler and evaluator. We conceptually separate out the role of **G/E** and **P_A/P_B** since parties swap garbling and evaluation roles in the dual execution protocol.

Table 1: Definitions of the notation and macros used in this paper.

Notation	Definitions
κ	Computational security parameter
ρ	Statistical security parameter
$x \leftarrow S$	Sample x uniformly at random from S
$[n]$	Zero-indexed set of n elements $\{0, 1, \dots, n-1\}$
$\mathbf{x}, x_i, \mathbf{X}, X_{i,j}$	Vector, the i -th entry of $\mathbf{x}$, matrix, the (i, j) -th entry of $\mathbf{X}$
λ_w, Λ_w	A bit masking the underlying value and the masked value on wire w
$r^*, \hat{r}$	A compressed bit r^* to be used in $\mathcal{F}_{\text{cpre}}$, share $\hat{r}$ of an AND triple
L_w, Λ_w	The label for wire index w , with masked value $\Lambda_w \in \{0, 1\}$
$\text{lsb}(x)$	Least significant bit of x
ExtBit	Procedure to extract a public bit from a label.
B2F	Macro to convert from $\mathbb{F}_2^\kappa$ to $\mathbb{F}_{2^\kappa}$
F2B	Macro to convert from $\mathbb{F}_{2^\kappa}$ to $\mathbb{F}_2^\kappa$
$\mathcal{C}, \mathcal{O}$	A Boolean circuit, the set of circuit-output wires in $\mathcal{C}$
$\mathcal{I}_A, \mathcal{I}_B$	The sets of circuit-input wires of P_A and P_B
$\mathcal{W}$	The set of output wires from non-free circuit gates
$\llbracket x \rrbracket_\Delta = (\llbracket x \rrbracket_\Delta^A, \llbracket x \rrbracket_\Delta^B)$	IT-MAC authentication of x under the global key Δ , P_A holds $\llbracket x \rrbracket_\Delta^A$, P_B holds $\llbracket x \rrbracket_\Delta^B$
Open ($\llbracket x \rrbracket_\Delta$)	Opening bit x authenticated by Δ
Garble , Eval	Garbling and evaluation of distributed garbling
Tensor.G , Tensor.E	Garbling and evaluation subroutines for tensor gates
$\mathcal{H}(x)$	One-hot encoding of the value x
$\mathcal{T}$	Truth table matrix used in tensor evaluation
$\top$	Matrix transpose symbol
id	Identity function used in tensor evaluations
Bad_y	Event that a consistency check fails (used in SFR definitions) on input y

2.2 Circuits with Tensor Gates

A boolean circuit $\mathcal{C}$ consists of a list of gates (i, j, k, T) , where i, j are indices of input wires, k is the index of output wire and $T \in \{\oplus, \wedge\}$ denotes the gate type. We evaluate a Boolean circuit in topological order as $v_k = v_i \oplus v_j$ if $T = \oplus$ and $v_k = v_i \wedge v_j$ if $T = \wedge$. In the 2PC setting, we use $\mathcal{I}_A$ (resp., $\mathcal{I}_B$) to denote the set of circuit input wire indices corresponding to the input of **P_A** (resp., **P_B**). We also use $\mathcal{W}$ to denote the set of output wire indices of all AND gates, and $\mathcal{O}$ to denote the set of circuit output wire indices in the circuit $\mathcal{C}$. We denote by $\mathcal{C}_{\text{and}}$ the set of all AND gates in the form of $(i, j, k, \wedge)$.

In this work, we consider Boolean circuits with tensor gates $\otimes$, which generalize AND gates to handle inputs of arbitrary length. Specifically, a circuit $\mathcal{C}$ consists of a list of gates in the form $(\alpha, \beta, \gamma, T)$ where α, β , are vectors of input wire indices of length n and m , respectively, γ is an $n \times m$ matrix of output wire indices, and $T \in \{\oplus, \otimes\}$. If $T = \oplus$, then $|\gamma| = |\alpha| = |\beta|$ and we have a standard XOR gate and if $T = \otimes$ then $|\gamma| = |\alpha| \cdot |\beta|$ and we compute the tensor of two boolean vectors as $v_{\gamma_{ij}} = v_{\alpha_i} v_{\beta_j}$ for $(i, j) \in [n] \times [m]$.

For a circuit $\mathcal{C}$ over tensor gates, we use $\mathcal{W}$ to denote the list of output wires of all $\otimes$ gates. This is consistent with how $\mathcal{W}$ is defined if $\mathcal{C}$ is first cast into a boolean circuit.

2.3 Hash Functions

Our work uses a circular correlation robust under naturally derived keys (ccrnd) hash function H_{ccrnd} [ZRE15]. Intuitively, a ccrnd produces random-looking output, even when hashing strings related by some correlation Δ , and even when using the output of the hash to encrypt strings that also involve the same correlation Δ . This enables the Free XOR technique [KS08a], which in turn is required for the half-gates [ZRE15] and one-hot garbling [HK21b] techniques which we build upon. We use the definition provided by [ZRE15]:

Definition 1 (Circular Correlation Robustness). *We define two oracles that each accept as input a label $K \in \{0, 1\}^\kappa$, a nonce i , and a bit b :*

- $\text{circ}_\Delta(K, i, b) \triangleq H(K \oplus \Delta, i) \oplus b \cdot \Delta$ where $\Delta \in \{0, 1\}^{\kappa-1} || 1$.
- $\mathcal{R}(K, i, b)$ is a random function with κ -bit output.

A sequence of oracle queries (K, i, b) is legal when the same value (K, i) is never queried with different values of b . H is circular correlation robust under naturally derived keys (ccrnd) if no poly-time adversary $\mathcal{A}$ issuing legal queries can distinguish circ_Δ and $\mathcal{R}$ i.e.

$$\left| \Pr_{\Delta} [\mathcal{A}^{\text{circ}_\Delta}(1^\kappa) = 1] - \Pr_{\mathcal{R}} [\mathcal{A}^{\mathcal{R}}(1^\kappa) = 1] \right| < \text{negl}(\kappa).$$

In practice, H_{ccrnd} is efficiently instantiated using fixed key AES [GKWY20].

2.4 Information-Theoretic Message Authentication Codes

We use information-theoretic message authentication codes (IT-MACs) [BDOZ11, NNOB12] to authenticate bits or field elements in $\mathbb{F}_{2^\kappa}$. Let $\Delta \in \mathbb{F}_{2^\kappa}$ be a *global key*. If this is known to P_A (resp. P_B) we denote it Δ_A (resp. Δ_B). We use $\llbracket x \rrbracket_\Delta = (\llbracket x \rrbracket_\Delta^A, \llbracket x \rrbracket_\Delta^B)$ to denote an element $x \in \mathbb{F}$ (where $\mathbb{F} \in \{\mathbb{F}_2, \mathbb{F}_{2^\kappa}\}$) shared between parties. Informally, holding IT-MACs under the garbler’s global key is required for garbling, holding IT-MACs under the evaluator’s global key is required for consistency checking. Note that the value x might be known to P_A or P_B or both or neither (secret-shared between them).

We recall the fact that IT-MACs are additively homomorphic and support public scalar multiplication. For a constant value $c \in \mathbb{F}_{2^\kappa}$, define $\llbracket c \rrbracket_\Delta = (c\Delta, 0)$. Given public coefficients $c_0, c_1, \dots, c_\ell$, two parties can locally compute $\llbracket y \rrbracket_\Delta := \llbracket c_0 \rrbracket_\Delta + \sum_{i=1}^\ell c_i \llbracket x_i \rrbracket_\Delta$.

We can extend these notions to authenticated vectors and matrices of bits in the natural manner. Let $m, \ell \in \mathbb{N}$ be dimension parameters. For a vector $\mathbf{x} \in \mathbb{F}_{2^\kappa}^\ell$, we denote by $\llbracket \mathbf{x} \rrbracket_\Delta = (\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta)$ a vector of authenticated values.

IT-MACs support “free” linear operations. In particular, given a matrix $\mathbf{M} \in \mathbb{F}^{m \times \ell}$, we denote by $\llbracket \mathbf{y} \rrbracket_\Delta = \mathbf{M} \cdot \llbracket \mathbf{x} \rrbracket_\Delta$ the matrix multiplication operation on authenticated vectors, where for each $i \in [m]$, $\llbracket y_i \rrbracket_\Delta = \sum_{j \in [\ell]} M_{i,j} \cdot \llbracket x_j \rrbracket_\Delta$.

2.4.1 Dual Authenticated Bits

Let Δ_A be the global key of P_A and Δ_B be the global key of P_B . Let $\Delta = (\Delta_A, \Delta_B, 1)$. In this case, we refer to $\llbracket x \rrbracket_\Delta$ as a *dual authenticated bit* as parties hold an XOR share of the bit x , which is authenticated under both P_A ’s and P_B ’s global key. We use these for garbling and consistency checking. Trivially, dual authenticated bits also support free linear operations.

2.4.2 Multiplication Triples

For $a, b, c \in \mathbb{F}_{2^\kappa}$, we refer to the tuple $\llbracket a \rrbracket_\Delta, \llbracket b \rrbracket_\Delta, \llbracket c \rrbracket_\Delta$ with $c = a \cdot b$ and $\Delta = (\Delta_A || \Delta_B || 1)$ as a *dual authenticated multiplication triple*. In the case where $a, b, c \in \mathbb{F}_2$, we also refer to the tuple as an AND triple. Prior works [WRK17, KRRW18, DILO22, CWYY23] instead use IT-MACs of form $\llbracket a_1 \rrbracket_{\Delta_B}, \llbracket b_1 \rrbracket_{\Delta_B}, \llbracket c_1 \rrbracket_{\Delta_B}, \llbracket a_2 \rrbracket_{\Delta_A}, \llbracket b_2 \rrbracket_{\Delta_A}, \llbracket c_2 \rrbracket_{\Delta_A}$ where $c_1 + c_2 = (a_1 + a_2)(b_1 + b_2)$ and P_A knows a_1, b_1, c_1 and P_B knows a_2, b_2, c_2 in the clear. We observe that it suffices (and seems more natural) to use dual authenticated multiplication triples instead. They are at least as easy to generate as the above which can be locally converted into a dual authenticated multiplication triple by defining $\llbracket a \rrbracket_\Delta^A = (a_1 \Delta_A + \llbracket a_2 \rrbracket_{\Delta_A}^A || \llbracket a_1 \rrbracket_{\Delta_B}^A || a_1)$ $\llbracket a \rrbracket_\Delta^B = (\llbracket a_2 \rrbracket_{\Delta_A}^B || a_2 \Delta_B + \llbracket a_1 \rrbracket_{\Delta_B}^B || a_2)$ and similarly for $\llbracket b \rrbracket_\Delta^A, \llbracket b \rrbracket_\Delta^B$ and $\llbracket c \rrbracket_\Delta^A, \llbracket c \rrbracket_\Delta^B$. It is straightforward to check that this gives us the desired relation.

2.4.3 Tensor Triples

For circuits with tensor gates (see Section 2.2) we also deal with vectors and matrices of wire indices. Therefore, we extend this notation to vectors and matrices in the natural manner. For the distributed garbling scheme that we present, we actually require preprocessed tensor triples $\llbracket \mathbf{a} \rrbracket_\Delta, \llbracket \mathbf{b} \rrbracket_\Delta, \llbracket \mathbf{c} \rrbracket_\Delta$ with $\mathbf{c} = \mathbf{a} \otimes \mathbf{b}$, which can be generated using $O(nm)$ AND triples where $n = |\mathbf{a}|, m = |\mathbf{b}|$. See Appendix E for more details.

2.4.4 Garbled Sharings

In our technical overview, we adopt the IT-MAC notation (which we refer to as *garbled sharings*) as a convenient way to abstract away the encryption “labels” used in standard GC protocols. The underlying GC mechanism, including round complexity, remains unchanged.

In the context of a garbling protocol, we use $\llbracket x^E \rrbracket$ (resp. $\llbracket x^G \rrbracket$) to denote that E (resp. G) knows the value of x in the clear.

We associate each wire w in the circuit with a semantic value z_w that is determined during evaluation. Typically, garbling protocols maintain the invariant that during evaluation parties hold *garbled sharings* corresponding to the (hidden) semantic value of all circuit wires. We explicitly refer to the individual components of such as garbled shares using $\llbracket z_w \rrbracket = (\mathbf{L}_{w,0}, \mathbf{L}_{w,z_w})$ where $\mathbf{L}_{w,1} = \mathbf{L}_{w,0} \oplus \Delta$. We extend this notation to vectors of wire indices $\mathbf{w} \in \{0,1\}^n$ where we define $\mathbf{L}_{\mathbf{w},\mathbf{b}}$ as $(\mathbf{L}_{w_i,b_i})_{i \in [n]}$ for $b_i \in \{0,1\}$. The GC sent by G is used by E to propagate her garbled shares of input wires $\{\mathbf{L}_{w,z_w}\}_{w \in \mathcal{I}}$ to garbled shares of output wires $\{\mathbf{L}_{w,z_w}\}_{w \in \mathcal{O}}$ of the circuit. The garbled sharings are then reconstructed for the circuit output wires only.

2.4.5 Batch opening of authenticated values

We review the procedure to perform batch opening of a vector of IT-MACs [NNOB12, DNNR17]. We assume that P_A holds the value in the clear and parties share IT-MACs. This procedure is defined symmetrically for the case of P_B holding the value. As in [CWYY23], we define the **CheckZero** procedure to check if all values are zero. Following [DNNR17], we hash IT-MAC components to reduce communication:

CheckZero($\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta$): On input authenticated values $\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta$, P_A convinces P_B that $x_i = 0$ for all $i \in [\ell]$ as follows:

1. P_A sends $h_A := H(\llbracket x_1 \rrbracket_\Delta^A, \dots, \llbracket x_\ell \rrbracket_\Delta^A)$ to P_B , where $H : \{0,1\}^* \rightarrow \{0,1\}^\kappa$ is a collision-resistant hash function.
2. P_B computes $h_B := H(\llbracket x_1 \rrbracket_\Delta^B, \dots, \llbracket x_\ell \rrbracket_\Delta^B)$ and checks that $h_A = h_B$. If the check fails, P_B aborts.

As in [CWYY23, DNNR17, WYKW21], we have the following lemmas, which we state for completeness and defer the reader to [CWYY23] for a proof.

Lemma 1 ([CWYY23, DNNR17, WYKW21]). *If H is collision-resistant hash function with soundness error ϵ , and if $\Delta \in \mathbb{F}_{2^\kappa}$ is sampled uniformly at random, then the probability that there exists $i \in [\ell]$ such that $x_i \neq 0$ and P_B accepts in the `CheckZero` procedure is bounded by $\frac{2}{2^\kappa} + \epsilon$.*

We define the batch-opening procedure, denoted by `Open`, for authenticated values $\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta$ over field $\mathbb{F}_{2^\kappa}$.

`Open`($\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta$): On input authenticated values $\llbracket x_1 \rrbracket_\Delta, \dots, \llbracket x_\ell \rrbracket_\Delta$, P_A opens as follows:

1. P_A sends purported openings $(x'_1, \dots, x'_\ell)$ to P_B .
2. Both parties compute $\llbracket y_i \rrbracket_\Delta := \llbracket x_i \rrbracket_\Delta + \llbracket x'_i \rrbracket_\Delta$ for each $i \in [\ell]$.
3. P_A and P_B invoke `CheckZero`($\llbracket y_1 \rrbracket_\Delta, \dots, \llbracket y_\ell \rrbracket_\Delta$). If P_B accepts, it outputs $(x'_1, \dots, x'_\ell)$.

Note that the communication complexity of `Open` is κ bits in total, amortized across all checked values. We use this for input encoding, consistency checking and output decoding in our final protocol $\Pi_{2\text{pc-tensor}}$.

2.4.6 Fixing authenticated bits

Random authenticated bits can be transformed into authenticated encodings of selected values using a postprocessing step denoted by `Fix` (also used in the DVZK protocol [DIO21, BMRS21]). Given $\llbracket \mathbf{r} \rrbracket_\Delta$ for $\mathbf{r} \in \{0, 1\}^\ell$ and an input $\mathbf{x} \in \{0, 1\}^\ell$, parties proceed as follows:

1. P_B sends $\mathbf{d} := \mathbf{x} + \mathbf{a}$ to P_A .
2. Both parties locally compute the authenticated encoding of $\mathbf{x}$ as:

$$\llbracket \mathbf{x} \rrbracket_\Delta := \llbracket \mathbf{a} \rrbracket_\Delta + \llbracket \mathbf{d} \rrbracket_\Delta.$$

We use this for input encoding in our final protocol $\Pi_{2\text{pc-tensor}}$.

2.5 (m, L) -Independent Matrix

We recall the notion of an (m, L) -independent matrix, which originates from [DK99] and has been applied in the context of authenticated garbling in [DILO22, CWYY23].

Definition 2 ([CWYY23]). *We say that a matrix $\mathbf{M} \in \mathbb{F}_2^{n \times L}$ is (m, L) -independent if any m rows of $\mathbf{M}$ are linearly independent.*

This property implies the following: if $\mathbf{M} \in \mathbb{F}_2^{n \times L}$ is (m, L) -independent and $\mathbf{b}^* \leftarrow \mathbb{F}_2^L$ is sampled uniformly at random, then the product $\mathbf{b} := \mathbf{M} \cdot \mathbf{b}^*$ yields an m -wise independent distribution.

Lemma 2 states that when

$$L = \left\lceil \rho + m \cdot \log \left(\frac{en}{m} \right) + \frac{\log m}{2} \right\rceil,$$

a uniformly random matrix $\mathbf{M} \leftarrow \mathbb{F}_2^{n \times L}$ satisfies (m, L) -independence with probability at least $1 - 2^{-\rho}$. We refer the reader to [CWYY23] for the proof of Lemma 2.

Lemma 2 ([CWYY23]). *Let $L = \left\lceil \rho + m \cdot \log \left(\frac{en}{m} \right) + \frac{\log m}{2} \right\rceil$, and let $\mathbf{M} \leftarrow \mathbb{F}_2^{n \times L}$ be chosen uniformly at random. Then $\mathbf{M}$ is (m, L) -independent except with probability $2^{-\rho}$.*

We include this definition and lemma as it is used in our compressed preprocessing functionality $\mathcal{F}_{\text{cpre}}$ (Appendix E).

2.6 One Hot Garbling

In this work, we make black-box use of the [HK21b]’s *privacy-free* semi-honest tensor product protocol. Naively computing a tensor product within a GC using AND gates would require $O(nm)$ ciphertexts and [HK21b]’s improvement over this of $O(m+n)$ ciphertexts can be attributed to their *one-hot garbling* technique which achieves a communication versus computation tradeoff that is concretely efficient in practice.

Let $\mathbf{x} \in \{0, 1\}^n$ and $\mathbf{y} \in \{0, 1\}^m$ be binary vectors, let $\mathcal{H}(\cdot)$ be the function that maps a bitstring to its one-hot encoding, and $\text{val}(\cdot)$ be the function that maps a binary vector $\mathbf{x} \in \{0, 1\}^n$ to its integer value $\text{val}(\mathbf{x}) \in [2^n]$. The one-hot encoding of a binary vector $\mathbf{x} \in \{0, 1\}^n$ is the length 2^n vector $\mathcal{H}(\mathbf{x})$ which is zero at all indices except at index $\text{val}(\mathbf{x})$, where it is one.

Definition 3 (One-Hot Encoding, [HK21b]). *Let $\mathbf{x} \in \{0, 1\}^n$ be a binary vector. Define $\text{val}(\mathbf{x}) \in [2^n]$ as the integer representation of $\mathbf{x}$, given by*

$$\text{val}(\mathbf{x}) = \sum_{j=0}^{n-1} x_j \cdot 2^j.$$

Then, the one-hot encoding of $\mathbf{x}$ is the vector $\mathcal{H}(\mathbf{x}) \in \{0, 1\}^{2^n}$ defined as:

$$\mathcal{H}(\mathbf{x})_i = \begin{cases} 1 & \text{if } i = \text{val}(\mathbf{x}) \\ 0 & \text{otherwise} \end{cases} \quad \text{for } i \in [2^n]$$

Using a GGM tree [GGM84] as described in [ZRE15], parties can use garbled shares $\llbracket x \rrbracket$ to compute garbled shares $\llbracket \mathcal{H}(\mathbf{x}) \rrbracket$ using only $2(n-1) + 1$ ciphertexts of communication, provided that E knows $\mathbf{x}$ in the clear. This technique, however, incurs exponential scaling in computation and is therefore restricted to small vectors. Formally, $n = O(\log \kappa)$. In fact, this can be extended to compute $\llbracket \mathcal{H}(\mathbf{x}) \otimes \mathbf{y} \rrbracket$ given $\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{y} \rrbracket$ for $\mathbf{x} \in \{0, 1\}^n, \mathbf{y} \in \{0, 1\}^m$ using $2(n-1) + m$ ciphertexts in total. [Hea24] improved this to $n-1+m$ ciphertexts in line with observations of [GYW⁺23]. Observe that the one-hot encoding $\mathcal{H}(\mathbf{x})$ is fully homomorphic: Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be an arbitrary function and let $\mathcal{T}(f) \in \{0, 1\}^{2^n \times m}$ denote its truth table, represented as a matrix. Then

$$\mathcal{T}(f)^\top \cdot \mathcal{H}(\mathbf{x}) = f(\mathbf{x}).$$

To obtain the value $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket$ using the one-hot tensor product $\llbracket \mathcal{H}(\mathbf{x}) \otimes \mathbf{y} \rrbracket$, we apply the identity $\mathcal{T}(\text{id})^\top \cdot (\mathcal{H}(\mathbf{x}) \otimes \mathbf{y}) = \mathbf{x} \otimes \mathbf{y}$ (Lemma 3, [HK21b]). This allows the parties to locally convert garbled shares of the one-hot tensor product to garbled shares of the standard tensor product using XOR homomorphism. Section 4 reformulates this protocol; we refer the reader to [HK21b] for more details.

3 Overview

This section presents our techniques in sufficient detail to understand our main contribution: an authenticated GC capable of handling tensor gates efficiently.

The key challenge in GC protocols can be viewed as follows: The garbler G is required to send material upfront that allows the evaluator E to propagate encoded inputs X to encoded outputs Y . For known GC schemes that use the FreeXOR [KS08b] optimization, values v are encoded as *garbled shares* $\llbracket v \rrbracket_{\Delta_{\mathsf{G}}}$ under a global key Δ_{G} that is known only to the garbler G . For circuits, this is done on a per-gate basis, but the number of protocol rounds stays constant: all garbled material is sent ahead of time. Indeed, all of G ’s shares are picked before inputs are known, so the garbled material really allows E to propagate its share of $\llbracket \mathbf{x} \rrbracket_{\Delta_{\mathsf{G}}}$ to its share of $\llbracket f(\mathbf{x}) \rrbracket_{\Delta_{\mathsf{G}}}$. FreeXOR allows E to compute $\llbracket u \oplus v \rrbracket_{\Delta_{\mathsf{G}}}$ given $\llbracket u \rrbracket_{\Delta_{\mathsf{G}}}, \llbracket v \rrbracket_{\Delta_{\mathsf{G}}}$. Therefore, the size garbled material scales with the number of non-XOR gates such as AND/tensor gates.

For privacy, such protocols also require that G introduces randomness $\mathbf{r}$ in the GC so that E does not learn the underlying value $\mathbf{x}$ but is still able to convert from $\llbracket \mathbf{x} \rrbracket_{\Delta_{\mathsf{G}}}$ to $\llbracket f(\mathbf{x}) \rrbracket_{\Delta_{\mathsf{G}}}$. To obtain security against a malicious garbler, it is necessary that (1) randomness be distributed between G and E and (2) E holds a commitment to this randomness so that the garbled material’s correctness can be checked. Authenticated

garbling exploits the fact that (1) many GC protocols only use $\mathbf{r}$ in the form of shares $\llbracket \mathbf{r} \rrbracket_{\Delta_G}$ which can be preprocessed without either party learning $\mathbf{r}$ and (2) $\mathbf{r}$ can be committed to $\mathbf{E}$ by parties preprocessing shares $\llbracket \mathbf{r} \rrbracket_{\Delta_E}$ where Δ_E is a second global key known only to $\mathbf{E}$. Since IT-MACs are linearly homomorphic, this leads to a very cheap post-evaluation correctness check. This moves all of the overhead incurred by malicious security to the preprocessing phase and recent protocols [DILO22, CWYY23] have discovered communication-efficient methods to preprocess these IT-MACs if $\mathbf{r}$ consists of degree-2 correlations of uniform bits, which turns out to be sufficient for garbling AND and Tensor gates.

With this context, our main challenge lies in constructing a maliciously secure protocol for garbling tensor gates using garbled shares $\llbracket \mathbf{r} \rrbracket_{\Delta_G}$ and homomorphic IT-MACs $\llbracket \mathbf{r} \rrbracket_{\Delta_E}$ of shared random bits $\mathbf{r}$. We conceptually separate this challenge into two parts: first, we present a novel protocol for garbling tensor gates using garbled shares of preprocessed random bits; second, we describe how we employ the dual execution technique [CWYY23] to check the correctness of garbled material post-evaluation using preprocessed homomorphic IT-MACs.

We present our first contribution in the format of distributed garbling [CWYY23] using two macros **Garble** and **Eval**, assuming that the preprocessed correlations $\llbracket \mathbf{r} \rrbracket_{\Delta_G}$ are already available. Looking ahead, this helps us simplify the security proof by abstracting properties of *selective-failure resistance* and *obliviousness*, which we discuss in Section 4. We construct a distributed garbling scheme ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) that uses *compressed* garbled shares of degree-2 correlations $\llbracket \mathbf{r} \rrbracket_{\Delta_G}$ to garble tensor gates with low communication. In section 3.1, we present the key ideas behind our construction and motivate them by reformulating prior garbling protocols: half-gates [ZRE15], distributed half-gates [KRRW18], and a generalization of half-gates to handle tensor (outer-product) gates [HK21b]. Our presentation of distributed half-gates differs from the original formulation, but we prove the equivalence in Appendix D.

Next, in Section 3.2, we give a high-level overview of the dual-execution technique [CWYY23] that we employ in our final protocol $\Pi_{2\text{pc-tensor}}$ for checking correctness of garbled material post-evaluation using linearly homomorphic IT-MAC commitments $\llbracket \mathbf{r} \rrbracket_{\Delta_E}$. The reason we use this technique is that it is compatible with compressed preprocessing in a way that makes black-box calls to our $\text{Garble}_{\text{Tensor}}$ and $\text{Eval}_{\text{Tensor}}$ macros. We analyze the communication complexity of this protocol in Section 5 and discuss applications in Appendix B. Finally, we have implemented and report on benchmarks of our maliciously secure tensor gate performance in Appendix C that demonstrate the concrete performance improvements of our approach compared to prior works.

3.1 Distributed Garbling with Tensor Gates

In this section, we review prior GC protocols, namely: semi-honest half-gates [ZRE15], distributed (malicious) half-gates [KRRW18], and a semi-honest generalization of half-gates to handle tensor (outer-product) gates [HK21b]. We describe how these techniques motivate our handling of distributed tensor gates. We reformulate these protocols (see Appendix D) so they more naturally lead to our distributed garbling scheme ($\text{Garble}_{\text{Tensor}}, \text{Garble}_{\text{Eval}}$). In this overview, we focus on the treatment of non-XOR gates as XOR gates can be computed locally using Free-XOR [KS08b]. For brevity, we use $\llbracket \cdot \rrbracket$ to denote $\llbracket \cdot \rrbracket_{\Delta_G}$ here.

3.1.1 Semi-Honest Half Gates

Seeking to reduce the size of garbled AND gates, [ZRE15] introduced the idea of a *garbler* (*resp. evaluator*) *half-gate* which allows parties to compute a privacy-free AND gate. The term “half-gate” refers to the fact that one party (either $\mathbf{G}$ or $\mathbf{E}$) knows the value of one input wire (out of two input wires) when evaluating the gate, allowing more efficient garbling. Two of these half-gates are then combined to achieve full privacy.

It was shown by [ZRE15] that an AND half-gate can be encoded using only one ciphertext of communication. In particular, there exist secure 2-party protocols GarbleAnd^E and GarbleAnd^G that can be used to compute $\llbracket x^E \rrbracket \cdot \llbracket y \rrbracket = \llbracket xy \rrbracket$ and $\llbracket x^G \rrbracket \cdot \llbracket y \rrbracket = \llbracket xy \rrbracket$ respectively. These two protocols are then combined [ZRE15] to compute $\llbracket x \rrbracket \cdot \llbracket y \rrbracket = \llbracket xy \rrbracket$ by rewriting xy as

$$x(\lambda \oplus \lambda \oplus y) = x\lambda \oplus x(\lambda \oplus y).$$

Here λ is the random bit sampled by G (also called a *mask*) and $\lambda \oplus y$ is revealed to E . Since G knows both λ and Δ , he can inject $\llbracket \lambda \rrbracket$ into the GC by setting his share to $\lambda\Delta$; E 's share is the all-zero string. By XOR-homomorphism parties can use this to compute $\llbracket \lambda \oplus y \rrbracket := \llbracket \lambda \rrbracket \oplus \llbracket y \rrbracket$. Parties use GarbleAnd^G for the first product $\llbracket x \rrbracket \cdot \llbracket \lambda^G \rrbracket = \llbracket x\lambda \rrbracket$ and GarbleAnd^E for the second one $\llbracket x \rrbracket \cdot \llbracket (\lambda \oplus y)^E \rrbracket = \llbracket x(\lambda \oplus y) \rrbracket$. Finally, again by XOR homomorphism, parties can compute $\llbracket xy \rrbracket = \llbracket x\lambda \rrbracket \oplus \llbracket x(\lambda \oplus y) \rrbracket$.

It was observed [KRRW18]¹ that two *evaluator half-gates* can also be combined into one privacy-preserving AND gate as follows:

$$(x \oplus \lambda_x)\lambda_y \oplus (y \oplus \lambda_y)x \oplus \lambda_x\lambda_y = xy,$$

using two GarbleAnd^E gates to compute $(x \oplus \lambda_x)\lambda_y$ and $(y \oplus \lambda_y)x$, and injecting $\lambda_x\lambda_y$ into the circuit.

Looking ahead, this method is better suited for viewing tensor gates [HK21b] as a generalization of half-gates.

This approach leads to the following semi-honest protocol for evaluating an AND gate $(x, y, z, \wedge)^2$ which has the same concrete cost as [ZRE15]:

Half Gates*

Inputs: G holds masks $\lambda_x, \lambda_y, \lambda_z \in \{0, 1\}$ and injects shares $\llbracket \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket, \llbracket \lambda_x\lambda_y \rrbracket, [\lambda_z]$. E holds masked values $x \oplus \lambda_x, y \oplus \lambda_y$. Parties hold shares $\llbracket x \rrbracket, \llbracket y \rrbracket$.

Procedure:

1. Parties locally compute $\llbracket x \oplus \lambda_x \rrbracket = \llbracket x \rrbracket \oplus \llbracket \lambda_x \rrbracket, \llbracket y \oplus \lambda_y \rrbracket = \llbracket y \rrbracket \oplus \llbracket \lambda_y \rrbracket$.
2. Parties compute

$$\begin{aligned} \llbracket (x \oplus \lambda_x)\lambda_y \rrbracket &= \text{GarbleAnd}(\llbracket x \oplus \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket), \\ \llbracket (y \oplus \lambda_y)x \rrbracket &= \text{GarbleAnd}(\llbracket y \oplus \lambda_y \rrbracket, \llbracket x \rrbracket) \end{aligned}$$

using 2κ bits of communication.

3. Parties locally compute

$$\llbracket xy \rrbracket = \llbracket (x \oplus \lambda_x)\lambda_y \rrbracket \oplus \llbracket (y \oplus \lambda_y)x \rrbracket \oplus \llbracket \lambda_x\lambda_y \rrbracket.$$

G sends $[xy] \oplus [\lambda_z]$ to E to reveal $xy \oplus \lambda_z$ (Remark 2).

Outputs: Parties hold garbled shares $\llbracket xy \rrbracket$, E holds masked value $xy \oplus \lambda_z$.

Remark 1. *Moving forward, we deal only with evaluator half-gates and therefore use GarbleAnd instead of GarbleAnd^E .*

Remark 2. *We set $\Delta = \Delta_G || 1$ so that for a garbled share $\llbracket x \rrbracket = (X, X \oplus x\Delta)$, parties also hold bit-XOR shares of x via $\text{lsb}(X) \oplus \text{lsb}(X \oplus x\Delta) = x$.*

3.1.2 Distributed Half Gates

It was observed [KRRW18] that the generation of garbled material in the half gates garbling scheme can be distributed between parties by preprocessing IT-MACs. We reformulate the KRRW distributed half-gates protocol [KRRW18] below and justify it in Appendix D.

¹The authors do not make this observation explicit but it is implied by their half-gates protocol (Section 3.1, [KRRW18]).

²In this overview, we overload the wire ID with its value to declutter notation.

Distributed Half-Gates*

Inputs: Parties preprocess shares $\llbracket \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket, \llbracket \lambda_x \lambda_y \rrbracket, \llbracket \lambda_z \rrbracket$. E holds masked values $x \oplus \lambda_x, y \oplus \lambda_y$. Parties hold shares $\llbracket x \rrbracket, \llbracket y \rrbracket$.

Procedure:

1. Parties locally compute $\llbracket x \oplus \lambda_x \rrbracket = \llbracket x \rrbracket \oplus \llbracket \lambda_x \rrbracket, \llbracket y \oplus \lambda_y \rrbracket = \llbracket y \rrbracket \oplus \llbracket \lambda_y \rrbracket$.
2. Parties compute

$$\begin{aligned} \llbracket (x \oplus \lambda_x) \lambda_y \rrbracket &= \text{GarbleAnd}(\llbracket x \oplus \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket), \\ \llbracket (y \oplus \lambda_y) x \rrbracket &= \text{GarbleAnd}(\llbracket y \oplus \lambda_y \rrbracket, \llbracket x \rrbracket) \end{aligned}$$

using 2κ bits of communication.

3. Parties locally compute

$$\llbracket xy \rrbracket = \llbracket (x \oplus \lambda_x) \lambda_y \rrbracket \oplus \llbracket (y \oplus \lambda_y) x \rrbracket \oplus \llbracket \lambda_x \lambda_y \rrbracket.$$

G sends $\llbracket xy \rrbracket \oplus \llbracket \lambda_z \rrbracket$ to E to reveal $xy \oplus \lambda_z$ (Remark 2).

Outputs: Parties hold garbled shares $\llbracket xy \rrbracket$, E holds masked value $xy \oplus \lambda_z$.

Observe that the *only* difference between this formulation of half-gates and distributed half-gates, is how garbled shares of masks are computed. Notice that the garbling procedure is, in fact, identical!

3.1.3 Tensor Gates

The garbled outer product (tensor) protocol [HK21b] can be viewed as a generalization of our modified half-gates protocol. As discussed in Section 2.6, the work of [HK21b] first shows how to compute $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket$ given garbled shares of vectors $\mathbf{x} \in \{0, 1\}^n, \mathbf{y} \in \{0, 1\}^m$ when E knows one input to the gate, using $2(n-1) + m$ ciphertexts of communication. We call this protocol **GarbleTensor** and observe that it exactly reduces to *evaluator half-gates* [ZRE15] (**GarbleAnd**) if we set $n = 1, m = 1$. Similar to the spirit of the above semi-honest half-gates protocol, [HK21b] shows how to evaluate a full tensor gate by invoking **GarbleTensor** twice. We present this protocol below.

Tensor Gates [HK21b]

Inputs: G holds masks $\lambda_x \in \{0, 1\}^n, \lambda_y \in \{0, 1\}^m, \lambda_z \in \{0, 1\}^{nm}$ and injects shares $\llbracket \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket, \llbracket \lambda_x \otimes \lambda_y \rrbracket, \llbracket \lambda_z \rrbracket$. E holds masked values $\mathbf{x} \oplus \lambda_x, \mathbf{y} \oplus \lambda_y$. Parties hold shares $\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{y} \rrbracket$.

Procedure:

1. Parties locally compute $\llbracket \mathbf{x} \oplus \lambda_x \rrbracket = \llbracket \mathbf{x} \rrbracket \oplus \llbracket \lambda_x \rrbracket, \llbracket \mathbf{y} \oplus \lambda_y \rrbracket = \llbracket \mathbf{y} \rrbracket \oplus \llbracket \lambda_y \rrbracket$.
2. Parties compute

$$\begin{aligned} \llbracket (\mathbf{x} \oplus \lambda_x) \otimes \lambda_y \rrbracket &= \text{GarbleTensor}(\llbracket \mathbf{x} \oplus \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket), \\ \llbracket (\mathbf{y} \oplus \lambda_y) \otimes \mathbf{x} \rrbracket &= \text{GarbleTensor}(\llbracket \mathbf{y} \oplus \lambda_y \rrbracket, \llbracket \mathbf{x} \rrbracket) \end{aligned}$$

using $(3(n+m) - 4)\kappa$ bits of communication.

3. Parties locally compute

$$\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket = \llbracket (\mathbf{x} \oplus \lambda_x) \otimes \lambda_y \rrbracket \oplus \llbracket (\mathbf{y} \oplus \lambda_y) \otimes \mathbf{x} \rrbracket \oplus \llbracket \lambda_x \otimes \lambda_y \rrbracket.$$

G sends $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket \oplus \llbracket \lambda_z \rrbracket$ to E to reveal $\mathbf{x} \otimes \mathbf{y} \oplus \lambda_z$ (Remark 2).

Outputs: Parties hold garbled shares $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket$, E holds masked value $\mathbf{x} \otimes \mathbf{y} \oplus \lambda_z$.

Note that the only change compared to half gates, other than syntactic ones, is that we replace **GarbleAnd** with **GarbleTensor**. Furthermore, the **GarbleTensor** construction [HK21b] that we use is a strict generalization of the **GarbleAnd** construction [ZRE15].

3.1.4 Distributed Tensor Gates

The changes made to our half-gates protocol to distribute the garbling process are disjoint from the changes made to handle tensor gates, and therefore can be composed together. Combining the changes i.e. (a) preprocessing garbled shares of random bits so they are not revealed to either party and (b) replacing **GarbleAnd** with **GarbleTensor** to efficiently handle tensor gates naturally gives rise to the following protocol, which we formalize in Section 4.2.

Distributed Tensor Gates

Inputs: Parties preprocess shares $\llbracket \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket, \llbracket \lambda_x \otimes \lambda_y \rrbracket, \llbracket \lambda_z \rrbracket$. E holds masked values $\mathbf{x} \oplus \lambda_x, \mathbf{y} \oplus \lambda_y$. Parties hold shares $\llbracket \mathbf{x} \oplus \lambda_x \rrbracket, \llbracket \mathbf{y} \oplus \lambda_y \rrbracket$.

Procedure:

1. Parties locally compute $\llbracket \mathbf{x} \oplus \lambda_x \rrbracket = \llbracket \mathbf{x} \rrbracket \oplus \llbracket \lambda_x \rrbracket, \llbracket \mathbf{y} \oplus \lambda_y \rrbracket = \llbracket \mathbf{y} \rrbracket \oplus \llbracket \lambda_y \rrbracket$.
2. Parties compute

$$\begin{aligned} \llbracket (\mathbf{x} \oplus \lambda_x) \otimes \lambda_y \rrbracket &= \text{GarbleTensor}(\llbracket \mathbf{x} \oplus \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket), \\ \llbracket (\mathbf{y} \oplus \lambda_y) \otimes \mathbf{x} \rrbracket &= \text{GarbleTensor}(\llbracket \mathbf{y} \oplus \lambda_y \rrbracket, \llbracket \mathbf{x} \rrbracket) \end{aligned}$$

using $(3(n + m) - 4)\kappa$ bits of communication.

3. Parties locally compute

$$\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket = \llbracket (\mathbf{x} \oplus \lambda_x) \otimes \lambda_y \rrbracket \oplus \llbracket (\mathbf{y} \oplus \lambda_y) \otimes \mathbf{x} \rrbracket^\top \oplus \llbracket \lambda_x \otimes \lambda_y \rrbracket.$$

G sends $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket \oplus \llbracket \lambda_z \rrbracket$ to E to reveal $\mathbf{x} \otimes \mathbf{y} \oplus \lambda_z$ (Remark 2).

Outputs: Parties hold garbled shares $\llbracket \mathbf{x} \otimes \mathbf{y} \rrbracket$, E holds masked value $\mathbf{x} \otimes \mathbf{y} \oplus \lambda_z$.

The key security properties that we require from a distributed garbling scheme to be able to compile it into a maliciously secure 2PC protocol are obliviousness (Definition 6) and selective-failure resistance (Definition 5). Informally, our scheme inherits obliviousness from [HK21b] as G's ciphertexts are computed using the one-hot garbling technique. This notion is formalized in Lemma 7. Our scheme is selective-failure resistant because the values revealed to E are masked, and the masks are secret-shared; thus, any abort does not leak information to G. This notion is formalized in Lemma 8.

3.2 Consistency Check via Dual Execution

In this section, we explain the dual execution technique at a high level which we employ in our final protocol $\Pi_{2\text{pc-tensor}}$ for checking correctness of garbled material post-evaluation using linearly homomorphic IT-MAC commitments $\llbracket \mathbf{r} \rrbracket_{\Delta_E}$ of the random bits used in garbling. We start by recapping the consistency check technique used in [KRRW18] to verify correctness of the garbled material in the distributed half-gates garbling scheme.

3.2.1 Proving correctness of masked triples

The distributed half-gates scheme [KRRW18] was originally used as follows:

1. Parties preprocess $\llbracket \lambda \rrbracket_\Delta$ for $\Delta = \Delta_G \parallel \Delta_E$ where λ consists of uniformly sampled random bits λ_w for each wire w and triples $(\lambda_x, \lambda_y, \lambda_x \lambda_y)$ for each AND gate $(x, y, z, \wedge)$ in the circuit. Conceptually, one can separate $\lambda = (\lambda_{\mathcal{I}}, \lambda_{\mathcal{W}}, \lambda_{\mathcal{O}})$ into randomness used for (1) input encoding, (2) garbling and consistency check and (3) output decoding.
2. Using $\llbracket \lambda_{\mathcal{W}} \rrbracket_{\Delta_G}$, parties locally run distributed garbling **Garble** to generate $(\mathcal{GC}_G, \mathcal{GC}_E)$.
3. Using $\llbracket \lambda_{\mathcal{I}} \rrbracket_\Delta$, G reveals masked inputs $\mathbf{x} \oplus \lambda_x, \mathbf{y} \oplus \lambda_y$ and shares of $\llbracket \mathbf{x} \rrbracket_{\Delta_G}, \llbracket \mathbf{y} \rrbracket_{\Delta_G}$ to E, encoding parties' private inputs in a secure and authenticated manner.
4. G send $\mathcal{GC}_G$ to E who runs **Eval** on masked inputs, input labels, $\mathcal{GC}_A$ and $\mathcal{GC}_B$ to obtain garbled shares $\llbracket z_w \rrbracket$ and masked values $z_w \oplus \lambda_w$ for every wire w in $\mathcal{C}$.
5. E reveals masked values $z_w \oplus \lambda_w$ for every wire in $\mathcal{C}$ to G. Using $\llbracket \lambda_{\mathcal{W}} \rrbracket_{\Delta_E}$ from preprocessing as homomorphic commitments to masks, G can now prove correctness of all evaluated values using a single hash. For every AND gate $(x, y, z, \wedge)$, the check ensures that $z = xy$. XOR gates are free and therefore do not need to be checked. If masked triples are correct for every gate, it follows by induction that the output is correct.
6. G then reveals output masks λ_w for $w \in \mathcal{O}$ to E who can check them against the commitments $\llbracket \lambda_{\mathcal{O}} \rrbracket_{\Delta_E}$ from preprocessing. E then unmask the output to obtain $\mathbf{z} = \mathcal{C}(\mathbf{x}, \mathbf{y})$ as required.

In this framework, the preprocessing phase is communication intensive, incurring an amortized cost of $O(\kappa)$ bits per AND gate. Our approach for garbling and checking correctness of tensor gates requires preprocessed tensor triples $\lambda_x, \lambda_y, \lambda_x \otimes \lambda_y$ which we generate in a black-box way from $O(nm)$ AND triples, $n = |\lambda_x|, m = |\lambda_y|$. If we were to use the above method for checking consistency, the total communication per tensor gate would be $O(nm)\kappa$, which overshoots our asymptotic target of $O(n + m)\kappa + O(nm)$ bits.

3.2.2 Compressed preprocessing

The above technique requires that E reveal all masked values to G so that it can prove correctness. As a result, we require that the masks are distributed uniformly at random. [DILO22] observed that if the masked wire values are not revealed to G, then E's wire masks need not be uniformly random. In fact, it suffices for the mask shares of E to be 2ρ -wise independent *as long as masked values remain hidden from G*. A 2ρ -wise independent vector can be obtained by expanding a short uniformly random vector with a public random matrix. The two parties now only need to authenticate a short vector and E's wire mask can be expanded using the homomorphism of IT-MAC, significantly reducing the communication of preprocessing. This approach is used in [CWYY23] to construct a compressed preprocessing protocol with $O(1)$ bits of communication per AND gate. However, as discussed above, this is not compatible with the approach of proving correctness of masked triples.

3.2.3 Proving correctness via dual execution

To address this, [CWYY23] employs the dual execution to authenticate obtained masked values without revealing masked values. We use a similar approach in our final protocol $\Pi_{2\text{pc-tensor}}$ and give a high level overview below.

1. Perform steps 1-4 above twice, with P_A and P_B swapping roles as garbler G and evaluator E between the two executions consistent inputs. The preprocessing protocol for $\lambda_{\mathcal{I}}, \lambda_{\mathcal{O}}$ remains the same but now E's share of $\lambda_{\mathcal{W}}$ is only 2ρ -wise independent, therefore cheaper to generate.

2. After both executions, the parties hold garbled shares $\llbracket z_w \rrbracket_{\Delta_A}$ and $\llbracket z'_w \rrbracket_{\Delta_B}$ for each wire w , corresponding to the first and second executions, respectively. P_B learns masked values $z_w \oplus \lambda_w$ from the first execution and P_A learns masked values $z'_w \oplus \lambda'_w$ from the second.
3. For each wire w in $\mathcal{C}$ (including inputs), we check that the values z_w and z'_w in two executions are equal. We perform the check by verifying $z_w \cdot (\Delta_A \oplus \Delta_B) = z'_w \cdot (\Delta_A \oplus \Delta_B)$. Since $\Delta_A \oplus \Delta_B$ is unknown to both P_A and P_B , the probability that $z_w \neq z'_w$ but the check passes is negligible. Using commitments to input encoding and garbling randomness $\llbracket \lambda_{\mathcal{I}} \rrbracket_{\Delta_B}, \llbracket \lambda_{\mathcal{W}} \rrbracket_{\Delta_B}, \llbracket \lambda'_{\mathcal{I}} \rrbracket_{\Delta_A}, \llbracket \lambda'_{\mathcal{W}} \rrbracket_{\Delta_A}$, garbled shares and masked values, parties can perform this check for all wire values by communicating a single hash output. Since one party is assumed to garble the circuit honestly and input consistency is checked, if wire values are equal across both executions, the output must be correct.³
4. Parties reveal output masks λ_w, λ'_w for $w \in \mathcal{O}$ to one another and check the received values against the commitments $\llbracket \lambda_{\mathcal{O}} \rrbracket_{\Delta_B}, \llbracket \lambda'_{\mathcal{O}} \rrbracket_{\Delta_A}$ from preprocessing. Parties then unmask the output to obtain $z = z' = \mathcal{C}(x, y)$ as required.

This way, the preprocessing cost is $O(1)$ bits per AND gate and we are able to preprocess tensor triples $\lambda_x, \lambda_y, \lambda_x \otimes \lambda_y$ with $O(nm)$ bits of communication, staying within our asymptotic target of $O(n + m)\kappa + O(nm)$ bits total communication.

We formally present our distributed tensor garbling construction ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) in Section 4, upgrade it to a constant-round maliciously secure 2PC protocol $\Pi_{2\text{pc-tensor}}$ via dual execution in Section 5 and provide a full proof of security in Appendix A.

4 Distributed Garbling with Tensor Gates

In this section, we recall the distributed garbling in the preprocessing model syntax put forward by [CWYY23]. This format assumes that parties hold global keys Δ_G, Δ_E , dual authenticated bits $\llbracket \lambda \rrbracket_{\Delta}$ and dual authenticated triples $\llbracket \hat{\lambda} \rrbracket_{\Delta}$ where $\Delta := \Delta_G \parallel \Delta_E \parallel 1$. We recall the definition of selective-failure resistance from [CWYY23] and introduce notions of correctness and obliviousness. Broadly speaking, selective-failure resistance means that even if a garbled circuit is adversarially generated, the event that an honest evaluator obtains incorrect outputs is independent of its private input. This property is required so that an evaluator's abort in the 2PC protocol can be simulated, i.e. an abort does not leak any sensitive information about the evaluator's private input. The notion of obliviousness is adapted from the semi-honest GC literature [BHR12] to the distributed setting, and states that the garbled material and encoded inputs do not reveal anything about the garbler's inputs. We present our construction ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) in the format of distributed garbling and prove that it satisfies the three properties above, which are used as lemmas in the security proof (Theorem 1) of our final protocol $\Pi_{2\text{pc-tensor}}$.

4.1 Distributed Garbling

A distributed garbling scheme in the preprocessing model [CWYY23] consists of two macros:

- $\text{Garble}(\mathcal{C})$: G and E perform local operations as follows:
 - G computes and outputs $(\mathcal{GC}_G, \{\mathcal{L}_{w,0}, \mathcal{L}_{w,1}\}_{w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}})$
 - E computes and outputs $\mathcal{GC}_E$.
- $\text{Eval}(\mathcal{C}, \mathcal{GC}_G, \mathcal{GC}_E, \{(\Lambda_w, \mathcal{L}_{w,x_w})\}_{w \in \mathcal{I}}) \rightarrow \{(\Lambda_w, \mathcal{L}_{w,z_w})\}_{w \in \mathcal{W} \cup \mathcal{O}}$.

We introduce a definition of correctness which says that the masked values should be correct with respect to the preprocessed bits and semantic wire values on the circuit during evaluation.

³We must check consistency for output wires of all non-free gates and not just circuit outputs, otherwise the protocol is susceptible to a *selective-failure attack*. Check Definition 5 and Lemma 8 for further discussion.

Definition 4 (correctness). Let $(\text{Garble}, \text{Eval})$ be a distributed garbling scheme for circuits in the preprocessing model. Let $\mathcal{C}$ be a circuit. Let $\mathbf{x} \in \{0, 1\}^{|\mathcal{I}|}$. The garbler $\mathbf{G}$ holds $\Delta_{\mathbf{G}}$, the evaluator $\mathbf{E}$ holds $\Delta_{\mathbf{E}}$ and both parties share $\llbracket \lambda \rrbracket_{\Delta}, \llbracket \hat{\lambda} \rrbracket_{\Delta}$ where $\Delta := \Delta_{\mathbf{G}} \parallel \Delta_{\mathbf{E}} \parallel 1$.

1. Compute $\text{Garble}(\mathcal{C})$ to output $(\mathcal{GC}_{\mathbf{G}}, \{\mathbf{L}_{w,0}, \mathbf{L}_{w,1}\}_{w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}})$ and $\mathcal{GC}_{\mathbf{E}}$.
2. Define $\Lambda_w = x_w \oplus \lambda_w$ for $w \in \mathcal{I}$ and compute $\text{Eval}(\mathcal{C}, \mathcal{GC}_{\mathbf{A}}, \mathcal{GC}_{\mathbf{B}}, \{(\Lambda_w, \mathbf{L}_{w,x_w})\}_{w \in \mathcal{I}})$ to output $\{(\Lambda_w, \mathbf{L}_{w,z_w})\}_{w \in \mathcal{W} \cup \mathcal{O}}$.

We say that $(\text{Garble}, \text{Eval})$ is correct if $\mathbf{z} = \mathcal{C}(\mathbf{x})$ for all circuits $\mathcal{C}$ and inputs $\mathbf{x}$ where $z_w := \Lambda_w \oplus \lambda_w$ for $w \in \mathcal{O}$.

Next, we recall the definition of ϵ -selective failure resistance (Definition 5, [CWYY23]).

Definition 5 (ϵ -selective failure resistance, [CWYY23]). Let $(\text{Garble}, \text{Eval})$ define a distributed garbling scheme in the preprocessing model. Let $\mathcal{C}$ be a circuit. The garbler $\mathbf{G}$ holds $\Delta_{\mathbf{G}}$, the evaluator $\mathbf{E}$ holds $\Delta_{\mathbf{E}}$ and both parties share $\llbracket \lambda \rrbracket_{\Delta}, \llbracket \hat{\lambda} \rrbracket_{\Delta}$ where $\Delta := \Delta_{\mathbf{G}} \parallel \Delta_{\mathbf{E}} \parallel 1$. Consider the following experiment induced by $\mathbf{E}$'s input $\mathbf{y} \in \{0, 1\}^{|\mathcal{I}_{\mathbf{E}}|}$ and adversary $\mathbf{G}$:

1. Receive $\mathcal{GC}_{\mathbf{G}}, \{\mathbf{L}_{w,0}, \mathbf{L}_{w,1}\}_{w \in \mathcal{I}_{\mathbf{B}}}$ and $\{\Lambda_w, \mathbf{L}_{w,x_w}\}_{w \in \mathcal{I}_{\mathbf{A}}}$ from $\mathbf{G}$.
2. $\mathbf{E}$ runs $\text{Garble}(\mathcal{C})$ to obtain $\mathcal{GC}_{\mathbf{E}}$, defines $\Lambda_w = y_w \oplus \lambda_w$ for $w \in \mathcal{I}_{\mathbf{B}}$ and computes

$$\{\Lambda_w, \mathbf{L}_{w,z_w}\}_{w \in \mathcal{W} \cup \mathcal{O}} = \text{Eval}(\mathcal{GC}_{\mathbf{G}}, \mathcal{GC}_{\mathbf{E}}, \{(\Lambda_w, \mathbf{L}_{w,z_w})\}_{w \in \mathcal{I}_{\mathbf{A}} \cup \mathcal{I}_{\mathbf{B}}})$$

to learn Λ_w for $w \in \mathcal{W}$. Define $x_w = \lambda_w \oplus \Lambda_w$ for $w \in \mathcal{I}_{\mathbf{A}}$.

3. Let z_w be the output on the wire $w \in \mathcal{W}$ when evaluating $\mathcal{C}$ on inputs $\mathbf{x}, \mathbf{y}$, and let $\text{Bad}_{\mathbf{y}}$ be the event that

$$\exists w \in \mathcal{W} : z_w \neq \lambda_w \oplus \Lambda_w.$$

We call a distributed garbling scheme ϵ -selective failure resilient if for all circuit $\mathcal{C}$ and all inputs $\mathbf{y}, \mathbf{y}'$ with $\mathbf{y} \neq \mathbf{y}'$,

$$|\Pr[\text{Bad}_{\mathbf{y}}] - \Pr[\text{Bad}_{\mathbf{y}'}]| \leq \epsilon.$$

[CWYY23] shows that the half-gates distributed garbling scheme $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ when instantiated with 2ρ -wise independent preprocessing has $2^{-\rho}$ -selective failure resistance. Looking ahead, we will show that $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$, when instantiated with $2\rho \log \kappa$ -wise independent preprocessing has $2^{-\rho}$ -selective failure resistance.

Next, we introduce a new definition which says that a distributed garbling scheme is *oblivious* if the garbled material, encoded inputs and decoding material do not reveal anything about the input.

Definition 6 (ϵ -obliviousness). Let $(\text{Garble}, \text{Eval})$ define a distributed garbling scheme in the preprocessing model. Let $\mathcal{C}$ be a circuit. The garbler $\mathbf{G}$ holds $\Delta_{\mathbf{G}}$, the evaluator $\mathbf{E}$ holds $\Delta_{\mathbf{E}}$ and both parties share $\llbracket \lambda \rrbracket_{\Delta}, \llbracket \hat{\lambda} \rrbracket_{\Delta}$ where $\Delta := \Delta_{\mathbf{G}} \parallel \Delta_{\mathbf{E}} \parallel 1$. Consider the following experiment induced by an algorithm $\mathcal{S}_{\text{obv}}$ playing the role of $\mathbf{G}$, input $\mathbf{x}$ and an adversary $\mathbf{E}$:

1. Sample a bit $b \in \{0, 1\}$.
2. If $b = 0$, sample

$$\mathcal{S}_{\text{obv}}(\mathcal{C}) \rightarrow (F, X)$$

3. Else if $b = 1$, sample

$$\text{Garble}(\mathcal{C}) \rightarrow (\mathcal{GC}_{\mathbf{G}}, \{\mathbf{L}_{w,0}, \mathbf{L}_{w,1}\}_{w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}})$$

Set $F = \mathcal{GC}_{\mathbf{G}}$, encode $X = \{x_w \oplus \lambda_w, \mathbf{L}_{w,x_w}\}_{w \in \mathcal{I}}$.

4. Send (F, X) to $\mathcal{A}$, receive guess b' .

Let $\text{Adv}_{\mathcal{A}, \mathbf{x}}$ denote the probability that $\mathcal{A}$ guesses $b' = b$ when the game uses input $\mathbf{x}$. We say that a distributed garbling scheme is ϵ -oblivious if there exists an algorithm $\mathcal{S}_{\text{obv}}$ such that for all adversaries $\mathcal{A}$ and all inputs $\mathbf{x}$ we have $\text{Adv}_{\mathcal{A}, \mathbf{x}} \leq 1/2 + \epsilon$.

Remark 3. Definition 6 is a straightforward extension of obliviousness for semi-honest garbling schemes [BHR12] to distributed garbling.

4.2 Distributed Garbling with Tensor Gates

With the appropriate definitions set up in the previous section, we are ready to present our distributed garbling scheme $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ that is equipped to handle tensor gates efficiently. As a warm up, we start by presenting distributed half-gates $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ [KRRW18] rephrased to match our notation and prove correctness, obliviousness, and selective-failure resistance. This sets the stage for our distributed tensor-gates scheme, which we subsequently present and prove to be correct, oblivious and selective failure resistant.

4.2.1 Warm Up: Distributed Half-Gates

We describe the distributed half-gates protocol $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ rewritten in our notation below. $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ takes as input a circuit over the basis $\{\oplus, \wedge\}$. We assume the garbler G holds Δ_G , the evaluator E holds Δ_E and both parties share $[\lambda]_\Delta, [\hat{\lambda}]_\Delta$ where $\Delta := \Delta_G || 1$ and $\hat{\lambda}$ consists of triples $(\lambda_\alpha, \lambda_\beta, \lambda_\gamma^* = \lambda_\alpha \lambda_\beta)$ for each AND gate $(\alpha, \beta, \gamma, \wedge)$. We refer to G 's (resp. E 's) share of $[\lambda]_\Delta$ as $[\lambda]_\Delta^G$ (resp. $[\lambda]_\Delta^E$).

Distributed Half Gates $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$

- $\text{Garble}_{\text{HG}}(\mathcal{C})$:

1. For each circuit input wire $w \in \mathcal{I}$, G samples $L_{w,0} \leftarrow \mathbb{F}_2^k$ and $L_{w,1} := L_{w,0} \oplus \Delta$.
2. Following the circuit's topological order, for each gate $G = (\alpha, \beta, \gamma, T)$:
 - If $T = \oplus$, G computes $L_{\gamma,0} := L_{\alpha,0} \oplus L_{\beta,0}$ and $L_{\gamma,1} := L_{\gamma,0} \oplus \Delta$.
 - If $T = \wedge$, G computes

$$L_{\alpha,1} := L_{\alpha,0} \oplus \Delta, \quad L_{\beta,1} := L_{\beta,0} \oplus \Delta,$$

$$G_{\gamma,0}^{(G)} := H(L_{\alpha,0} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus H(L_{\alpha,1} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus [\lambda_\beta]_\Delta^G$$

$$G_{\gamma,1}^{(G)} := H(L_{\beta,0} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus H(L_{\beta,1} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus L_{\alpha,0},$$

$$L_{\gamma,0} := H(L_{\alpha,0} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus H(L_{\beta,0} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus [\lambda_\gamma^*]_\Delta^G,$$

$$\text{sets } \Lambda_\gamma^{(G)} := \text{ExtBit}(L_{\gamma,0}) \oplus \text{ExtBit}([\lambda_\gamma]_\Delta^G) \text{ and } L_{\gamma,1} = L_{\gamma,0} \oplus \Delta.$$

3. G outputs

$$\{L_{w,0}, L_{w,1}\}_{w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}} \quad \text{and} \quad \mathcal{GC}_G = \{G_{w,0}^{(G)}, G_{w,1}^{(G)}, \Lambda_w^{(G)}\}_{w \in \mathcal{W}}.$$

4. For each $(\alpha, \beta, \gamma, \wedge) \in \mathcal{W}$, E computes

$$G_{\gamma,0}^{(E)} := [\lambda_\beta]_\Delta^E$$

$$G_{\gamma,1}^{(E)} := 0$$

$$\Lambda_\gamma^{(E)} := \text{ExtBit}([\lambda_\gamma]_\Delta^E)$$

5. E outputs

$$\mathcal{GC}_E = \{G_{w,0}^{(E)}, G_{w,1}^{(E)}, \Lambda_w^{(E)}\}_{w \in \mathcal{W}}.$$

- $\text{Eval}(\mathcal{C}, \mathcal{GC}_G, \mathcal{GC}_E, \{(\Lambda_w, L_{w,z_w})\}_{w \in \mathcal{I}})$:

1. Following the circuit's topological order, for each gate $G = (\alpha, \beta, \gamma, T)$:

- If $T = \oplus$, E computes $\Lambda_\gamma = \Lambda_\alpha \oplus \Lambda_\beta$ and $L_{\gamma, z_\gamma} := L_{\alpha, z_\alpha} \oplus L_{\beta, z_\beta}$.
- If $T = \wedge$, E computes

$$G_{\gamma, 0} := G_{\gamma, 0}^{(G)} \oplus G_{\gamma, 0}^{(E)}, \quad G_{\gamma, 1} := G_{\gamma, 1}^{(G)} \oplus G_{\gamma, 1}^{(E)}$$

$$\begin{aligned} L_{\gamma, z_\gamma} := & H(L_{\alpha, z_\alpha} \oplus [\lambda_\alpha]_\Delta^E, \gamma) \oplus \Lambda_\alpha(G_{\gamma, 0}) \\ & \oplus H(L_{\beta, z_\beta} \oplus [\lambda_\beta]_\Delta^E, \gamma) \oplus \Lambda_\beta(G_{\gamma, 1} \oplus L_{\alpha, z_\alpha}) \\ & \oplus [\lambda_\gamma]_\Delta^E \end{aligned}$$

then set the public bit

$$\Lambda_\gamma := \Lambda_\gamma^{(G)} \oplus \Lambda_\gamma^{(E)} \oplus \text{ExtBit}(L_{\gamma, z_\gamma}).$$

2. E outputs $\{\Lambda_w, L_{w, z_w}\}_{w \in \mathcal{W} \cup \mathcal{O}}$.

Lemma 3. $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ is correct.

Proof. We prove this by induction on gates of the circuit. As correctness is trivial for $\oplus$ gates, we focus on $\wedge$ gates. Recall the half-gate strategy: If G holds A, B, Δ and E holds $A \oplus a\Delta, B \oplus b\Delta$ and a they can engage in the following procedure: G sets $C = H(A)$ and sends $G = H(A) \oplus H(A \oplus \Delta)$ to E who computes $H(A \oplus a\Delta) \oplus a(G \oplus B \oplus b\Delta) = C \oplus ab\Delta$ as required. Observe that

$$\begin{aligned} G_{\gamma, 0} &= H(L_{\alpha, 0} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus H(L_{\alpha, 1} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus \lambda_\beta \Delta \\ G_{\gamma, 1} &= H(L_{\beta, 0} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus H(L_{\beta, 1} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus L_{\alpha, 0}. \end{aligned}$$

As a result,

$$\begin{aligned} H(L_{\alpha, z_\alpha} \oplus [\lambda_\alpha]_\Delta^E, \gamma) \oplus \Lambda_\alpha(G_{\gamma, 0}) &= H(L_{\alpha, 0} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus \Lambda_\alpha \lambda_\beta \Delta, \\ H(L_{\beta, z_\beta} \oplus [\lambda_\beta]_\Delta^E, \gamma) \oplus \Lambda_\beta(G_{\gamma, 1} \oplus L_{\alpha, z_\alpha}) &= H(L_{\beta, 0} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus \Lambda_\beta z_\alpha \Delta \end{aligned}$$

using $\Lambda_\alpha = z_\alpha \oplus \lambda_\alpha$ and $\Lambda_\beta = z_\beta \oplus \lambda_\beta$.

Therefore,

$$\begin{aligned} L_{\gamma, z_\gamma} &= L_{\gamma, 0} \oplus (\Lambda_\alpha \lambda_\beta \oplus \Lambda_\beta z_\alpha) \Delta \oplus [\lambda_\gamma]_\Delta^G \oplus [\lambda_\gamma]_\Delta^E \\ &= L_{\gamma, 0} \oplus (\Lambda_\alpha \lambda_\beta \oplus \Lambda_\beta z_\alpha) \Delta \oplus \lambda_\alpha \lambda_\beta \Delta \\ &= L_{\gamma, 0} \oplus z_\alpha z_\beta \Delta \end{aligned}$$

as required. Finally, $\Lambda_\gamma = \text{ExtBit}(L_{\gamma, z_\gamma}) \oplus \text{ExtBit}(L_{\gamma, 0}) \oplus \text{ExtBit}(\lambda_\gamma \Delta) = \text{ExtBit}((z_\gamma \oplus \lambda_\gamma) \Delta) = z_\gamma \oplus \lambda_\gamma$ as required since $\text{ExtBit}(\Delta) = 1$ by definition. $\square$ $\square$

Lemma 4. $(\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}})$ is ϵ_{ccrnd} -oblivious.

Proof. We prove this by construction of a simulator $\mathcal{S}_{\text{obv}}(\mathcal{C})$. $\mathcal{S}_{\text{obv}}$ samples labels $\{L_w\}_{w \in \mathcal{I}}$ uniformly at random and defines $X = \{\lambda_w, L_w\}_{w \in \mathcal{I}}$. For each AND gate, $\mathcal{S}_{\text{obv}}$ samples $G_{\gamma, 0}^{(G)}, G_{\gamma, 1}^{(G)} \leftarrow \{0, 1\}^\kappa$ and $\Lambda_\gamma^{(G)} \leftarrow \{0, 1\}$ uniformly at random and concatenates these to form F . The input masked values and labels are indistinguishable since they are sampled in the same way (the masks λ_w are uniformly random to $\mathcal{A}$). Recall that $\text{Garble}_{\text{HG}}$ generates G 's garbled material for an AND gate $(\alpha, \beta, \gamma, \wedge)$ as follows:

$$\begin{aligned} G_{\gamma, 0}^{(G)} &:= H(L_{\alpha, 0} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus H(L_{\alpha, 1} \oplus [\lambda_\alpha]_\Delta^G, \gamma) \oplus [\lambda_\beta]_\Delta^G \\ G_{\gamma, 1}^{(G)} &:= H(L_{\beta, 0} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus H(L_{\beta, 1} \oplus [\lambda_\beta]_\Delta^G, \gamma) \oplus L_{\alpha, 0}, \end{aligned}$$

By the circular correlation robustness under naturally derived keys property of H_{ccrnd} , $G_{\gamma, 0}^{(G)}$ and $G_{\gamma, 1}^{(G)}$ sampled in this way are indistinguishable from uniform, except with ϵ_{ccrnd} probability. Next, recall that masked values

$\{\Lambda_\gamma\}_{\gamma \in \mathcal{W}}$ obtained during evaluation are uniformly distributed in $\mathcal{A}$'s view (as they are masked using random shares) and $\Lambda_\gamma^{(G)}$ is sampled in **Garble** such that $\Lambda_\gamma = \Lambda_\gamma^{(G)} \oplus \Lambda_\gamma^{(E)} \oplus \text{ExtBit}(\mathbf{L}_{\gamma, z_\gamma})$ so $\Lambda_\gamma^{(G)}$ is also uniformly distributed. $\square$

Lemma 5. ($\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}}$) is $2^{-\rho}$ -selective failure resistant when $\mathbf{E}$'s masks are 2ρ -wise independent.

Proof. Lemma 8, [CWYY23]. $\square$

4.2.2 Distributed Tensor Gates

We present our distributed tensor gates protocol ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) below. It takes as input circuits over the basis $\{\oplus, \otimes\}$ (see Section 2.2). We require that inputs to the tensor gates are $O(\log \kappa)$ bits long as computation complexity of one-hot garbling scales exponentially in the size of tensor gate input [HK21b]. We discuss how to handle arbitrary length tensor products in Appendix B. When tensor gates are restricted to input vectors of length 1, we exactly recover the distributed half gates protocol ($\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}}$). We assume the garbler $\mathbf{G}$ holds $\Delta_{\mathbf{G}}$, the evaluator $\mathbf{E}$ holds $\Delta_{\mathbf{E}}$ and both parties share $[\![\lambda]\!]_{\Delta}, [\![\hat{\lambda}]\!]_{\Delta}$ where $\Delta := \Delta_{\mathbf{G}} || 1$ and $\hat{\lambda}$ consists of triples $(\lambda_\alpha, \lambda_\beta, \lambda_\gamma^* = \lambda_\alpha \otimes \lambda_\beta)$ for each Tensor gate $(\alpha, \beta, \gamma, \otimes)$. We refer to $\mathbf{G}$'s (resp. $\mathbf{E}$'s) share of $[\![\lambda]\!]_{\Delta}$ as $[\![\lambda]\!]_{\Delta}^{\mathbf{G}}$ (resp. $[\![\lambda]\!]_{\Delta}^{\mathbf{E}}$).

Distributed Tensor Gates ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$)

- $\text{Garble}_{\text{Tensor}}(\mathcal{C})$:

1. For each circuit input wire $w \in \mathcal{I}$, sample $\mathbf{L}_{w,0} \leftarrow \mathbb{F}_2^\kappa$ and $\mathbf{L}_{w,1} := \mathbf{L}_{w,0} \oplus \Delta$.
2. Following the circuit's topological order, for each gate $G = (\alpha, \beta, \gamma, T)$:
 - If $T = \oplus$, $\mathbf{G}$ computes $\mathbf{L}_{\gamma,0} := \mathbf{L}_{\alpha,0} \oplus \mathbf{L}_{\beta,0}$ and $\mathbf{L}_{\gamma,1} := \mathbf{L}_{\gamma,0} \oplus \Delta$.
 - If $T = \otimes$, let $n = |\alpha|, m = |\beta|$. Then $\mathbf{G}$ computes

$$\begin{aligned} (\mathbf{Z}_{\gamma,0}, \mathbf{G}_{\gamma,0}^{(\mathbf{G})}) &:= \text{Tensor.Gb}(n, m, \Delta, \mathbf{L}_{\alpha,0} \oplus [\![\lambda_\alpha]\!]_{\Delta}^{\mathbf{G}}, [\![\lambda_\beta]\!]_{\Delta}^{\mathbf{G}}) \\ (\mathbf{Z}_{\gamma,1}, \mathbf{G}_{\gamma,1}^{(\mathbf{G})}) &:= \text{Tensor.Gb}(m, n, \Delta, \mathbf{L}_{\beta,0} \oplus [\![\lambda_\beta]\!]_{\Delta}^{\mathbf{G}}, \mathbf{L}_{\alpha,0}) \\ \mathbf{L}_{\gamma,0} &:= \mathbf{Z}_{\gamma,0} \oplus \mathbf{Z}_{\gamma,1}^\top \oplus [\![\lambda_\gamma^*]\!]_{\Delta}^{\mathbf{G}}, \end{aligned}$$

sets $\Lambda_\gamma^{(\mathbf{G})} := \text{ExtBit}(\mathbf{L}_{\gamma,0}) \oplus \text{ExtBit}([\![\lambda_\gamma]\!]_{\Delta}^{\mathbf{G}})$ and $\mathbf{L}_{\gamma,1} = \mathbf{L}_{\gamma,0} \oplus \Delta$.

3. $\mathbf{G}$ outputs

$$\{\mathbf{L}_{w,0}, \mathbf{L}_{w,1}\}_{w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}} \quad \text{and} \quad \mathcal{GC}_{\mathbf{G}} = \{\mathbf{G}_{\gamma,0}^{(\mathbf{G})}, \mathbf{G}_{\gamma,1}^{(\mathbf{G})}, \Lambda_\gamma^{(\mathbf{G})}\}_{\gamma \in \mathcal{W}}.$$

4. For each $(\alpha, \beta, \gamma, \otimes) \in \mathcal{W}$, $\mathbf{E}$ computes

$$\begin{aligned} \mathbf{G}_{\gamma,0}^{(\mathbf{E})} &:= [\![\lambda_\beta]\!]_{\Delta}^{\mathbf{E}} \\ \mathbf{G}_{\gamma,1}^{(\mathbf{E})} &:= \mathbf{0} \\ \Lambda_\gamma^{(\mathbf{E})} &:= [\![\lambda_\gamma]\!]_{\Delta}^{\mathbf{E}} \end{aligned}$$

5. $\mathbf{P}_{\mathbf{B}}$ outputs

$$\mathcal{GC}_{\mathbf{E}} = \{\mathbf{G}_{\gamma,0}^{(\mathbf{E})}, \mathbf{G}_{\gamma,1}^{(\mathbf{E})}, \Lambda_\gamma^{(\mathbf{E})}\}_{\gamma \in \mathcal{W}}.$$

- $\text{Eval}(\mathcal{C}, \mathcal{GC}_{\mathbf{G}}, \mathcal{GC}_{\mathbf{E}}, \{(\Lambda_w, \mathbf{L}_{w, z_w})\}_{w \in \mathcal{I}})$:

1. Following the circuit's topological order, for each gate $G = (\alpha, \beta, \gamma, T)$:
 - If $T = \oplus$, $\mathbf{E}$ computes $\Lambda_\gamma = \Lambda_\alpha \oplus \Lambda_\beta$ and $\mathbf{L}_{\gamma, \Lambda_\gamma} := \mathbf{L}_{\alpha, \Lambda_\alpha} \oplus \mathbf{L}_{\beta, \Lambda_\beta}$.
 - If $T = (\otimes, n, m)$, $\mathbf{E}$ computes

$$\begin{aligned} \mathbf{Z}_{\gamma,0} &:= \text{Tensor.Ev}(n, m, \Lambda_\alpha, \mathbf{G}_{\gamma,0}^{(\mathbf{G})}, \mathbf{L}_{\alpha, z_\alpha}, \mathbf{G}_{\gamma,0}^{(\mathbf{E})}) \\ \mathbf{Z}_{\gamma,1} &:= \text{Tensor.Ev}(m, n, \Lambda_\beta, \mathbf{G}_{\gamma,1}^{(\mathbf{G})}, \mathbf{L}_{\beta, z_\beta}, \mathbf{L}_{\alpha, z_\alpha} \oplus \mathbf{G}_{\gamma,1}^{(\mathbf{E})}) \\ \mathbf{L}_{\gamma, z_\gamma} &:= \mathbf{Z}_{\gamma,0} \oplus \mathbf{Z}_{\gamma,1}^\top \oplus [\![\lambda_\gamma^*]\!]_{\Delta}^{\mathbf{E}} \end{aligned}$$

then set the public bit

$$\mathbf{A}_\gamma := \mathbf{A}_\gamma^{(G)} \oplus \mathbf{A}_\gamma^{(E)} \oplus \text{ExtBit}(\mathbf{L}_\gamma, z_\gamma).$$

2. $\mathbf{E}$ outputs $\{\mathbf{A}_w, \mathbf{L}_{w, z_w}\}_{w \in \mathcal{W} \cup \mathcal{O}}$

For conciseness, we use the **Tensor** macro which is a reformulation of the privacy-free garbled outer product protocol from [HK21b].

Tensor macro

Tensor.Gb($n, m, \Delta, \mathbf{A}, \mathbf{B}$):

1. Interpret $\mathbf{A} = A_0 A_1 \cdots A_{n-1}$. Create a PRG tree with 2^n leaves labeling the j -th node at the i -th level $S_{i,j}$ using $S_{0,0} = A_0 \oplus \Delta$ and $S_{0,1} = A_0$ as root seeds for the left and right subtrees respectively.

2. Define

$$G_{i,0} = H(A_i \oplus \Delta, \nu_{i,0}) \oplus \bigoplus_{j=0}^{2^i-1} S_{i,2j}, \quad G_{i,1} = H(A_i, \nu_{i,1}) \oplus \bigoplus_{j=0}^{2^i-1} S_{i,2j+1}$$

for $i \in [n-1]$ where $\nu_{i,b}$ is a one-time nonce. Set $\mathbf{G} = \{G_{i,0}, G_{i,1}\}_{i \in [n-1]}$.

3. Let $\mathbf{L}_\ell$ for $\ell \in [2^n]$ denote the leaf seeds. Using a PRG, expand each leaf seed $\mathbf{L}_\ell$ into m pseudorandom values $X_{\ell,k}$ for $k \in [m]$.

4. Interpret $\mathbf{B} = B_0 B_1 \cdots B_{m-1}$. Define

$$G_k = \left(\bigoplus_{\ell=0}^{2^n-1} X_{\ell,k} \right) \oplus B_k$$

for $k \in [m]$. Append $\mathbf{G} = \mathbf{G} || \{G_k\}_{k \in [m]}$.

5. Define $\mathbf{X}_G = (X_{\ell,k})_{\ell \in [2^n], k \in [m]}$. Let $\text{id} : \{0,1\}^n \rightarrow \{0,1\}^n$ denote the identify function. Set $\mathbf{Z}_G = \mathcal{T}(\text{id})^\top \cdot \mathbf{X}_G$.

Output $(\mathbf{Z}_G, \mathbf{G})$

Tensor.Ev($n, m, \mathbf{a}, \mathbf{G}, \mathbf{A} \oplus \mathbf{a}\Delta, \mathbf{B} \oplus \mathbf{b}\Delta$):

1. Use $A_0 \oplus \mathbf{a}_0 \Delta$ to reproduce subtree with root node $S_{0,\bar{\mathbf{a}}_0}$.

2. For $i \in [n-1]$ use $A_i \oplus \mathbf{a}_i \Delta$, $G_{i,\bar{\mathbf{a}}_i}$ and previously obtained nodes $S_{i,j}$ to decrypt $S_{i,a_0 \cdots a_{i-1} \bar{\mathbf{a}}_i}$ and reproduce the subtree with that node as the root.

3. The above step reproduces $\mathbf{L}_\ell$ for $\ell \neq \mathbf{a}$. Using $\mathbf{L}_\ell$ for $\ell \neq \mathbf{a}$ as a root, create a PRG tree of seeds $X_{\ell,k}$ for $k \in [m]$.

4. Construct

$$X_{\mathbf{a},k} = \left(\bigoplus_{\ell \neq \mathbf{a}} X_{\ell,k} \right) \oplus G_k \oplus (B_k \oplus \mathbf{b}_k \Delta)$$

for $k \in [m]$.

5. Define $\mathbf{X}_E = (X_{\ell,k})_{\ell \in [2^n], k \in [m]}$. Let $\text{id} : \{0,1\}^n \rightarrow \{0,1\}^n$ denote the identify function. Set $\mathbf{Z}_E = \mathcal{T}(\text{id})^\top \cdot \mathbf{X}_E$.

Output $\mathbf{Z}_E$

Correctness: By correctness of one-hot gates [HK21b], $\mathbf{Z}_E = \mathbf{Z}_G \oplus (\mathbf{a} \otimes \mathbf{b})\Delta$.

Lemma 6. ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) is correct.

Proof. We prove this by induction on gates of the circuit. Correctness is trivial for $\oplus$ gates so we focus on $\otimes$ gates.

1. Recall that by the correctness of `Tensor` if $(Z, G) = \text{Tensor.Gb}(|X|, |Y|, \Delta, X, Y)$ then $\text{Tensor.Ev}(|X|, |Y|, i, G, X \oplus i\Delta, Y \oplus j\Delta) = Z \oplus (i \otimes j)\Delta$.
2. Next observe that $\mathbf{L}_{\alpha,0} \oplus \mathbf{L}_{\alpha,z_\alpha} \oplus [\lambda_\alpha]_\Delta^G \oplus [\lambda_\alpha]_\Delta^E = \Lambda_\alpha \Delta$ and $[\lambda_\beta]_\Delta^G \oplus [\lambda_\beta]_\Delta^E = \lambda_\beta \Delta$. Therefore, $Z_{\gamma,0}^{(E)} = Z_{\gamma,0}^{(G)} \oplus (\Lambda_\alpha \otimes \lambda_\beta) \Delta$.
3. Similarly, observe that $\mathbf{L}_{\beta,0} \oplus \mathbf{L}_{\beta,z_\beta} \oplus [\lambda_\beta]_\Delta^G \oplus [\lambda_\beta]_\Delta^E = \Lambda_\beta \Delta$ and $\mathbf{L}_{\alpha,0} \oplus \mathbf{L}_{\alpha,z_\alpha} = z_\alpha \Delta$. Therefore, $Z_{\gamma,1}^{(E)} = Z_{\gamma,1}^{(G)} \oplus (\Lambda_\beta \otimes z_\alpha) \Delta$.
4. As a result, $\mathbf{L}_{\gamma,0} \oplus \mathbf{L}_{\gamma,\Lambda_\gamma} = (\Lambda_\alpha \otimes \lambda_\beta) \Delta \oplus (\Lambda_\beta \otimes z_\alpha) \Delta \oplus (\lambda_\alpha \otimes \lambda_\beta) \Delta = z_\gamma \Delta$ as required.
5. Finally, $\Lambda_\gamma = \text{ExtBit}(\mathbf{L}_{\gamma,z_\gamma}) \oplus \text{ExtBit}(\mathbf{L}_{\gamma,0}) \oplus \text{ExtBit}(\lambda_\gamma \Delta) = \text{ExtBit}((z_\gamma \oplus \lambda_\gamma) \Delta) = z_\gamma \oplus \lambda_\gamma$ as required since $\text{ExtBit}(\Delta) = 1$ by definition.

□

□

Lemma 7. $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ is ϵ_{ccrnd} -oblivious.

Proof. We prove this by construction of $\mathcal{S}_{\text{obv}}(\mathcal{C})$. $\mathcal{S}_{\text{obv}}$ samples labels $\{\mathbf{L}_w\}_{w \in \mathcal{I}}$ uniformly at random and defines $X = \{\lambda_w, \mathbf{L}_w\}_{w \in \mathcal{I}}$. For a tensor gate $(\alpha, \beta, \gamma, \otimes)$, let $n = |\alpha|, m = |\beta|$. $\mathcal{S}_{\text{obv}}$ samples $\mathbf{G}_{\gamma,0}^{(G)} \leftarrow \{0, 1\}^{(2(n-1)+m)\kappa}$, $\mathbf{G}_{\gamma,1}^{(G)} \leftarrow \{0, 1\}^{(2(m-1)+n)\kappa}$ and $\Lambda_\gamma^{(G)} \leftarrow \mathbb{F}_2^{nm}$ uniformly at random. The input masked values and labels are indistinguishable since they are sampled in the same way (the masks λ_w are uniformly random to $\mathcal{A}$). Recall that `GarbleTensor` generates garbled material for a tensor gate by invoking `Tensor.Gb` to generate ciphertexts of the form $G_{i,0} = \text{H}_{\text{ccrnd}}(A_i \oplus \Delta_A, \nu_{i,0}) \oplus \bigoplus_{j=0}^{2^i-1} S_{i,2j}$ and $G_{i,1} = \text{H}_{\text{ccrnd}}(A_i, \nu_{i,1}) \oplus \bigoplus_{j=0}^{2^i-1} S_{i,2j+1}$ for each level of the GGM tree. Note that each string $S_{i,j}$ is generated by invoking `Hccrnd` on another uniform string and both ciphertexts are generated by again invoking `Hccrnd`. By the circular correlation-robust under naturally derived keys property of `Hccrnd`, $\mathcal{A}$ cannot distinguish these ciphertexts from random, except with ϵ_{ccrnd} probability. Next, recall that masked values $\{\Lambda_\gamma\}_{\gamma \in \mathcal{W}}$ obtained during evaluation are uniformly distributed in $\mathcal{A}$'s view (as they are masked using random shares) and $\Lambda_\gamma^{(G)}$ is sampled in `Garble` such that $\Lambda_\gamma := \Lambda_\gamma^{(G)} \oplus \Lambda_\gamma^{(E)} \oplus \text{ExtBit}(\mathbf{L}_{\gamma,z_\gamma})$ so $\Lambda_\gamma^{(G)}$ is also uniformly distributed. □

Lemma 8. $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ is $2^{-\rho}$ -selective failure resistant when $\mathcal{E}$'s masks are $2\rho \log \kappa$ -wise independent.

Proof. We need to show that the abort probability of an honest $\mathcal{E}$ in the presence of a malicious $\mathcal{G}$ is independent of $\mathcal{E}$'s private input $\mathbf{y}$. Informally, this is because $\mathcal{E}$'s inputs to `Tensor.Ev` that depend on $\mathbf{y}$ are masked from $\mathcal{G}$ with sufficient entropy. Each incorrectly garbled ciphertext used by the evaluator to traverse the GGM tree for tensor gates has a $1/2$ probability of causing an abort, determined by the masked value $\Lambda_w = z_w \oplus \lambda_w$ used to traverse that step of the tree. If masks λ_w are distributed uniformly at random, then this abort is independent of the wire's semantic value and therefore simulatable. We argue that even if they are not distributed uniformly at random, but rather $2\rho \log \kappa$ -wise independent, then either the abort still looks random to $\mathcal{G}$ (and is therefore simulatable) or $\mathcal{G}$ is forced to correctly guess $2\rho \log \kappa$ mask values, which happens with probability at most $2^{-\rho}$. Observe the equation $z_w \oplus \lambda_w = \Lambda_w$. Fix an arbitrary $y \in \mathbb{F}_2^{|\mathcal{Z}|}$, and analyze the event $\text{Bad}_{\mathbf{y}}$. We define another event $\text{Bad}'_{\mathbf{y}}$ that there exists some $\gamma \in \mathcal{W}$ such that $(\Lambda_\gamma \oplus \lambda_\gamma) \neq (\Lambda_\alpha \oplus \lambda_\alpha) \cdot (\Lambda_\beta \oplus \lambda_\beta)$ i.e. a wire value is inconsistent with respect to its two predecessor wires. We argue this event is identical to $\text{Bad}_{\mathbf{y}}$. This is because if $z_w = \lambda_w \oplus \Lambda_w$ for all $w \in \mathcal{W}$ (it is true for $w \in \mathcal{I}$ by definition) then the above reduces to $z_\gamma = z_\alpha \cdot z_\beta$ which is true by definition. Conversely, assume the above relationship holds for all wires $w \in \mathcal{W}$. Inductively, assume that for wires α, β we have $z_\alpha = \lambda_\alpha \oplus \Lambda_\alpha$ and $z_\beta = \lambda_\beta \oplus \Lambda_\beta$ (the base case is true for input wires by definition). Then $z_\gamma = z_\alpha \cdot z_\beta = (\lambda_\alpha \oplus \Lambda_\alpha) \cdot (\lambda_\beta \oplus \Lambda_\beta) = (\lambda_\gamma \oplus \Lambda_\gamma)$ as required.

Now we analyze $\text{Bad}'_{\mathbf{y}}$. Consider the following pebbling game, where we view every wire w as a node in a DAG, and place a pebble on that node once the wire label and masked value for that wire is known.

Specifically, we place a blue pebble if the masked value of that wire is always correct, and a red pebble if the probability that the wire value is inconsistent with respect to its two predecessor wires (denote this event as $\text{Bad}'_{\mathbf{y},w}$) is non-zero.

Initially, we can place blue pebbles on all input wires, since they are correct by definition.

As an inductive step, given the preprocessing information and garbler's share of the garbled circuit $\mathcal{GC}_A$ (possibly malformed) and the evaluator's share $\mathcal{GC}_B$, we pebble output wires of gates whose input wires are already pebbled. If the gate is an XOR gate, we place a blue pebble. For a tensor gate $(\alpha, \beta, \gamma, \otimes)$ do as follows. Note that each masked input bit is used by the evaluator to decide between one of two received ciphertexts to decrypt to reconstruct seeds for that level of the GGM tree. We place a red pebble on an output wire if there are errors on either of the ciphertexts that affected the value of this output wire; if there are no errors *regardless of the choice of Λ_i, Λ_j* , we place a blue pebble. This information can be extracted by directly observing the garbled material and checking its consistency with respect to other ciphertexts and preprocessed bits. In particular, this check does not depend on E 's input $\mathbf{y}$. By definition, inputs to tensor gates have length less than $\log \kappa$ so each of P_A 's incorrectly garbled ciphertext can affect at most $\log \kappa$ output wires and therefore result in at most $\log \kappa$ red pebbles being placed.

Inductively, we can go through the entire DAG until all nodes are pebbled. Notice that the event $\text{Bad}'_{\mathbf{y}}$ occurs if the event $\text{Bad}'_{\mathbf{y},w}$ occurs on any of the nodes. Recall we set $\sigma = 2\rho \log \kappa$. Let ℓ be the number of red pebble nodes and consider the following two cases.

- $\ell \leq \rho \log \kappa$: In this case the $(2\rho \log \kappa, L)$ -independence of M ensures that the Λ_w values underlying all the $\text{Bad}'_{\mathbf{y},w}$ events are uniformly random to G , so the probability of $\text{Bad}'_{\mathbf{y}}$ is independent of the evaluator's input.
- $\ell > \rho \log \kappa$: Recall that each of G 's incorrectly garbled ciphertext can cause at most $\log \kappa$ pebbles to be placed. Therefore, in this case at least ρ ciphertexts have been incorrectly garbled by G so $\text{Bad}'_{\mathbf{y}}$ not happening implies that ρ consecutive coin flips all come up heads, which occurs with probability $2^{-\rho}$.

Therefore, for different evaluator inputs $\mathbf{y}$ and $\mathbf{y}'$, the probabilities of $\text{Bad}_{\mathbf{y}}$ and $\text{Bad}_{\mathbf{y}'}$ differ by at most $2^{-\rho}$. In other words, the distributed tensor garbling scheme with $2\rho \log \kappa$ -wise independent compressed preprocessing is $2^{-\rho}$ -selective failure resilient. $\square$ $\square$

5 Authenticated Garbling with Tensor Gates

In this section, we present our final protocol $\Pi_{2\text{pc-tensor}}$ that employs the dual execution technique. We prove that it is a maliciously secure 2PC protocol when the underlying distributed garbling is $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ from the previous section. $\Pi_{2\text{pc-tensor}}$ garbles a tensor gate $G = (\alpha, \beta, \gamma, \otimes)$ with $O(n + m)\kappa + O(nm)$ where bits of communication where $n = |\alpha|, m = |\beta|$ and $n, m = O(\log \kappa)$.

The correctness of our 2PC protocol follows immediately from the correctness of the underlying distributed garbling scheme and our correctness check which passes if parties are honest (Lemma 9). We state our main security theorem below, deferring its proof to Appendix A. We use $\mathcal{F}_{\text{cpre}}$ (Appendix E) to instantiate compressed preprocessing. This is a minor modification of the compressed preprocessing functionality presented in [CWYY23] and we refer the reader to the original paper for more details.

Protocol $\Pi_{2\text{pc-tensor}}$

Inputs: Parties agree on a circuit $\mathcal{C}$ with input wires $\mathcal{I}$, output wires of all non-XOR gates $\mathcal{W}$ and circuit-output wires $\mathcal{O}$. P_A holds $\mathbf{x} \in \{0, 1\}^{|\mathcal{I}_A|}$, P_B holds $\mathbf{y} \in \{0, 1\}^{|\mathcal{I}_B|}$ and P_B receives output $\mathbf{z} = \mathcal{C}(\mathbf{x}, \mathbf{y})$. Let $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ denote the distributed tensor garbling procedure.

Execution 1: P_A plays the role of a garbler and P_B acts as an evaluator.

Preprocess:

1. P_A and P_B call $\mathcal{F}_{\text{cpre}}$ to get a matrix $\mathbf{M}$ and IT-MACs $([\mathbf{a}]_{\Delta_B}, [\hat{\mathbf{a}}]_{\Delta_B}, [\mathbf{b}^*]_{\Delta_A}, [\hat{\mathbf{b}}]_{\Delta_A})$. They locally

compute $\llbracket \mathbf{b} \rrbracket_{\Delta_A} = \mathbf{M} \cdot \llbracket \mathbf{b}^* \rrbracket_{\Delta_A}$. The locally convert IT-MACs to dual authenticated bits $\llbracket \lambda \rrbracket_{\Delta}, \llbracket \hat{\lambda} \rrbracket_{\Delta}$ where $\lambda := a \oplus b$ and $\Delta = \Delta_A \parallel \Delta_B \parallel 1$ (see Section 2.4).

Garble:

2. P_A and P_B run **Garble** to generate a distributed garbled circuit $(\mathcal{GC}_A, \mathcal{GC}_B)$. Further, for each wire w , P_A holds two labels $L_{w,0}, L_{w,1} \in \{0, 1\}^\kappa$ satisfying $L_{w,1} = L_{w,0} \oplus \Delta_A$.

Encode Inputs:

3. Define $\lambda_A = (\lambda_w)_{w \in \mathcal{I}_A}$. Parties holding $\llbracket \lambda_A \rrbracket_{\Delta}$ reveal λ_A to P_A via **Open** $(\llbracket \lambda_A \rrbracket_{\Delta_A})$. Parties call **Fix** $(\mathbf{x}, \llbracket \lambda_A \rrbracket_{\Delta_A})$ revealing $\Lambda_w = x_w \oplus \lambda_w$ and L_{w,x_w} for $w \in \mathcal{I}_A$ to P_B .
4. Define $\lambda_B = (\lambda_w)_{w \in \mathcal{I}_B}$. Parties holding $\llbracket \lambda_B \rrbracket_{\Delta}$ reveal λ_B to P_B via **Open** $(\llbracket \lambda_B \rrbracket_{\Delta_B})$. Parties call **Fix** $(\mathbf{y}, \llbracket \lambda_B \rrbracket_{\Delta_B})$ revealing $\Lambda_w = y_w \oplus \lambda_w$ and L_{w,y_w} for $w \in \mathcal{I}_B$ to P_B .

Evaluate:

5. P_A sends $\mathcal{GC}_A$ to P_B .
6. P_B runs **Eval** $(\mathcal{GC}_A, \mathcal{GC}_B, \{(\Lambda_w, L_{w,z_w})\}_{w \in \mathcal{I}_A \cup \mathcal{I}_B})$ to obtain (Λ_w, L_{w,z_w}) for $w \in \mathcal{W} \cup \mathcal{O}$. For $w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}$, parties define $\llbracket z_w \rrbracket_{\Delta_A} = (L_{w,0}, L_{w,z_w})$ and they hold $\llbracket \lambda_w \rrbracket_{\Delta_B}$ from preprocessing.

Execution 2: Swapping roles, P_B plays the role of a garbler and P_A acts as an evaluator. For $w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}$, parties obtain $\llbracket z'_w \rrbracket_{\Delta_B} = (L'_{w,0}, L'_{w,z'_w})$ and $\llbracket \lambda'_w \rrbracket_{\Delta_A}$. P_A additionally holds Λ'_w .

Consistency Check: P_A and P_B check that $(\Lambda_w \oplus \lambda_w) \cdot (\Delta_A \oplus \Delta_B) = (\Lambda'_w \oplus \lambda'_w) \cdot (\Delta_A \oplus \Delta_B)$ holds for $w \in \mathcal{I} \cup \mathcal{W}$

1. For each $w \in \mathcal{I} \cup \mathcal{W}$ the parties respectively compute

$$\begin{aligned} V_w^A &= \llbracket z_w \rrbracket_{\Delta_A}^A \oplus \llbracket z'_w \rrbracket_{\Delta_B}^A \oplus \llbracket \lambda_w \rrbracket_{\Delta_B}^A \oplus \llbracket \lambda'_w \rrbracket_{\Delta_A}^A \oplus (z'_w \oplus \lambda'_w) \Delta_A \\ V_w^B &= \llbracket z_w \rrbracket_{\Delta_A}^B \oplus \llbracket z'_w \rrbracket_{\Delta_B}^B \oplus \llbracket \lambda_w \rrbracket_{\Delta_B}^B \oplus \llbracket \lambda'_w \rrbracket_{\Delta_A}^B \oplus (z_w \oplus \lambda_w) \Delta_B \end{aligned}$$

2. P_A computes $h_A := H(\{V_w^A\}_{w \in \mathcal{W}})$ and sends it to P_B , who computes $h_B := H(\{V_w^B\}_{w \in \mathcal{W}})$ and checks $h_A = h_B$. If the check fails, P_B aborts.

Decode Outputs: For each $w \in \mathcal{O}$, parties run **Open** $(\llbracket \lambda_w \rrbracket_{\Delta_B})$ revealing z_w to P_B such that P_B learns $z_w = \Lambda_w \oplus \lambda_w$.

Theorem 1. $\Pi_{2\text{pc-tensor}}$ securely realizes functionality $\mathcal{F}_{2\text{pc}}$ in the presence of a static malicious adversary in the $\mathcal{F}_{\text{cpre}}$ -hybrid model.

Proof. Deferred to Appendix A. □

5.0.1 Communication complexity:

Ignoring the cost of input and output processing which is independent of the circuit size and concretely small, P_A and P_B send each other $\sum_{(\alpha, \beta, \gamma, \otimes) \in \mathcal{C}_{\text{tensor}}} 3((|\alpha| + |\beta|) - 4)\kappa + |\mathcal{W}|$ bits. Therefore the amortized online one-way communication is $3((n + m) - 4)\kappa + mn$ bits per $(\otimes, n, m)$ gate. Since we need to call $\mathcal{F}_{\text{cpre}}$ twice in $\Pi_{2\text{pc-tensor}}$, we conclude that the amortized one-way (resp. two-way) communication is $3((n + m) - 4)\kappa + 5mn$ (resp. $6((n + m) - 4)\kappa + 10mn$) bits, improving over prior state of the art of $2mn\kappa + 5mn$ bits. When handling tensor gates of arbitrary length (see Appendix B) we obtain a $O(\log \kappa)$ factor improvement in communication complexity over state of the art.

References

- [AH25] Jorge Aparicio and Brook Heisler. criterion.rs: Statistics-driven micro-benchmarking library, 2014–2025.
- [App17] Benny Applebaum. Garbled circuits as randomized encodings of functions: a primer. Cryptology ePrint Archive, Report 2017/385, 2017.
- [BDOZ11] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. pages 169–188, 2011.
- [BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. pages 784–796, 2012.
- [BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). pages 503–513, 1990.
- [BMR16] Marshall Ball, Tal Malkin, and Mike Rosulek. Garbling gadgets for Boolean and arithmetic circuits. pages 565–577, 2016.
- [BMRS21] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac’n’Cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*, pages 92–122, Cham, 2021. Springer International Publishing.
- [CWYY23] Hongrui Cui, Xiao Wang, Kang Yang, and Yu Yu. Actively secure half-gates with minimum overhead under duplex networks. pages 35–67, 2023.
- [DILO22] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Improving line-point zero knowledge: Two multiplications for the price of one. pages 829–841, 2022.
- [DIO21] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-Point Zero Knowledge and Its Applications. In Stefano Tessaro, editor, *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*, volume 199 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:24, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [DK99] Yevgeniy Dodis and Sanjeev Khanna. Space-time tradeoffs for graph properties. In Jiri Wiedermann, Peter van Emde Boas, and Mogens Nielsen, editors, *Automata, Languages and Programming*, pages 291–300, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [DNNR17] Ivan Damgård, Jesper Buus Nielsen, Michael Nielsen, and Samuel Ranellucci. The TinyTable protocol for 2-party secure computation, or: Gate-scrambling revisited. pages 167–187, 2017.
- [GGM84] O. Goldreich, S. Goldwasser, and S. Micali. How to construct randolli functions. In *25th Annual Symposium on Foundations of Computer Science, 1984.*, pages 464–479, 1984.
- [GKWY20] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. pages 825–841, 2020.
- [GTRS25] Tianyao Gu, Afonso Tinoco, Sri Harish G. Rajan, and Elaine Shi. PicoGRAM: Practical garbled RAM from decisional diffie-hellman. pages 247–281, 2025.
- [GYW⁺23] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. pages 330–362, 2023.
- [Hea24] David Heath. Efficient arithmetic in garbled circuits. pages 3–31, 2024.

- [HK20] David Heath and Vladimir Kolesnikov. Stacked garbling - garbled circuit proportional to longest execution path. pages 763–792, 2020.
- [HK21a] David Heath and Vladimir Kolesnikov. One hot garbling. pages 574–593, 2021.
- [HK21b] David Heath and Vladimir Kolesnikov. LogStack: Stacked garbling with $O(b \log b)$ computation. pages 3–32, 2021.
- [HKN24] David Heath, Vladimir Kolesnikov, and Lucien K. L. Ng. Garbled circuit lookup tables with logarithmic number of ciphertexts. pages 185–215, 2024.
- [HKO22] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. EpiGRAM: Practical garbled RAM. pages 3–33, 2022.
- [HKO23] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. Tri-state circuits - A circuit model that captures RAM. pages 128–160, 2023.
- [KN25] Vladimir Kolesnikov and Lucien Ng. Toss: Garbled PIR from table-only stacking. In *ACM CCS*, 2025.
- [KNR⁺17] Vladimir Kolesnikov, Jesper Buus Nielsen, Mike Rosulek, Ni Trieu, and Roberto Trifiletti. DUPLO: Unifying cut-and-choose for garbled circuits. pages 3–20, 2017.
- [KRRW18] Jonathan Katz, Samuel Ranellucci, Mike Rosulek, and Xiao Wang. Optimizing authenticated garbling for faster secure two-party computation. pages 365–391, 2018.
- [KS08a] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free xor gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *Automata, Languages and Programming*, pages 486–498, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [KS08b] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. pages 486–498, 2008.
- [LO13] Steve Lu and Rafail Ostrovsky. How to garble RAM programs. pages 719–734, 2013.
- [LP04] Yehuda Lindell and Benny Pinkas. A proof of yao’s protocol for secure two-party computation. Cryptology ePrint Archive, Paper 2004/175, 2004.
- [LP15] Yehuda Lindell and Benny Pinkas. An efficient protocol for secure two-party computation in the presence of malicious adversaries. 28(2):312–350, April 2015.
- [NNOB12] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. pages 681–700, 2012.
- [PLS23] Andrew Park, Wei-Kai Lin, and Elaine Shi. NanoGRAM: Garbled RAM with $\tilde{O}(\log N)$ overhead. pages 456–486, 2023.
- [RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. pages 94–124, 2021.
- [WGC19] Sameer Wagh, Divya Gupta, and Nishanth Chandran. SecureNN: 3-party secure computation for neural network training. 2019(3):26–49, July 2019.
- [WRK17] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Authenticated garbling and efficient maliciously secure two-party computation. pages 21–37, 2017.

- [WTB⁺21] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. Falcon: Honest-majority maliciously secure framework for private deep learning. 2021(1):188–208, January 2021.
- [WYKW21] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. pages 1074–1091, 2021.
- [ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. pages 220–250, 2015.

A Proof of Theorem 1

Lemma 9. Let $e_w := (\lambda_w \oplus \Lambda_w) \oplus (\lambda'_w \oplus \Lambda'_w)$ be the error on wire $w \in \mathcal{I} \cup \mathcal{W}$ after the execution of $\Pi_{2\text{pc-tensor}}$. Then the checking values V_w^A, V_w^B satisfy $V_w^A \oplus V_w^B = e_w \cdot (\Delta_A \oplus \Delta_B)$.

Proof. This is shown by a straightforward calculation.

$$\begin{aligned} V_w^A \oplus V_w^B &= z_w \Delta_A \oplus z'_w \Delta_B \oplus \lambda_w \Delta_B \oplus \lambda'_w \Delta_A \oplus (z'_w \oplus \lambda'_w) \Delta_A \oplus (z_w \oplus \lambda_w) \Delta_B \\ &= (z_w \oplus \lambda'_w \oplus z'_w \oplus \lambda'_w) \Delta_A \oplus (z'_w \oplus \lambda_w \oplus z_w \oplus \lambda_w) \Delta_B \\ &= (z_w \oplus z'_w) (\Delta_A \oplus \Delta_B) \\ &= e_w (\Delta_A \oplus \Delta_B). \end{aligned}$$

□

Theorem 2 (Theorem 1 restated). $\Pi_{2\text{pc-tensor}}$ securely realizes functionality $\mathcal{F}_{2\text{pc}}$ in the presence of a static malicious adversary in the $\mathcal{F}_{\text{cpre}}$ -hybrid model.

Proof. The protocol is symmetric in roles of P_A and P_B , except that the output is revealed to P_B . We prove security for the case of a corrupted P_B as the other case is easier and follows immediately. The running time of $\mathcal{A}$ is bounded by $\tau = \text{poly}(\kappa)$. We first describe the simulator $\mathcal{S}_B$ and then argue its effectiveness through a series of hybrid experiments.

Simulator $\mathcal{S}_B$ for malicious P_B

Recall that $\mathcal{S}_B$ is required to play the role of an honest P_A without knowing its input $\mathbf{x}$. At a high-level, $\mathcal{S}_B$ works as follows: In the first execution, it invokes $\mathcal{S}_{\text{obv}}$ to simulate garbling and cheats to open corresponding masks to $\mathcal{A}$ during input encoding, using input $\mathbf{0}$. In the second execution, on receiving $\mathcal{A}$'s garbled circuit, $\mathcal{S}_B$ extracts the errors e_w introduced on output wires of non-free gates. This is used to simulate the consistency check. It also extracts $\mathcal{A}$'s input $\mathbf{y}$ across both executions and uses this to compute the desired output $\mathbf{z} = \mathcal{C}(\mathbf{x}, \mathbf{y})$ by invoking $\mathcal{F}_{2\text{pc}}$. It cheats in output decoding by opening masks such that $\mathcal{A}$ obtains $\mathbf{z}$ if it behaves honestly, even though the simulated garbling corresponds to an input of $\mathbf{x} = \mathbf{0}$ in the first execution. Importantly, even in the second execution, $\mathcal{S}_B$ uses input $\mathbf{x} = \mathbf{0}$. This doesn't change the event that there exists an error $e_w \neq 0$ by selective-failure resistance of $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ (Lemma 8), and therefore the probability of abort during consistency check is unchanged.

We now formalize the simulator $\mathcal{S}_{\text{obv}}$ and prove indistinguishability of the real and ideal world via a hybrid argument.

Execution 1

Preprocess: During the simulation of $\mathcal{F}_{\text{cpre}}$, $\mathcal{S}_B$ receives from $\mathcal{A}$ a global key Δ_B and data to compute its dual authenticated bits $\llbracket \lambda \rrbracket_{\Delta}^B, \llbracket \hat{\lambda} \rrbracket_{\Delta}^B$ and reverse samples Δ_A and $\llbracket \lambda \rrbracket_{\Delta}^A, \llbracket \hat{\lambda} \rrbracket_{\Delta}^A$ where $\Delta := \Delta_A || \Delta_B || 1$.

Garble: $\mathcal{S}_B$ invokes $\mathcal{S}_{\text{obv}}$ to simulate $F = \mathcal{GC}_A, X = \{\Lambda_w, \mathsf{L}_w\}_{w \in \mathcal{I}}$.

Encode Inputs: $\mathcal{S}_B$ simulates **Open** and **Fix** implicitly using input $\mathbf{x} = \mathbf{0}$ to reveal masked inputs and labels $\{\Lambda_w, \mathsf{L}_w\}_{w \in \mathcal{I}}$ and extracts $\mathcal{A}$'s input $\mathbf{y}$.

Evaluate: $\mathcal{S}_B$ sends $\mathcal{GC}_A$ to $\mathcal{A}$. $\mathcal{S}_B$ also computes $\mathcal{GC}_B$ and runs $\text{Eval}(\mathcal{C}, \mathcal{GC}_A, \mathcal{GC}_B, \{(\Lambda_w, \mathsf{L}_w)\}_{w \in \mathcal{I}})$ to obtain $\llbracket z_w \rrbracket_{\Delta_A}^B$ and $z_w \oplus \lambda_w$ for $w \in \mathcal{W} \cup \mathcal{O}$, the values that an honest $\mathcal{A}$ would obtain after evaluation.

Execution 2

Preprocess: During the simulation of $\mathcal{F}_{\text{cpre}}$, $\mathcal{S}_B$ receives from $\mathcal{A}$ data to compute its dual authenticated bits $[\![\lambda']\!]_{\Delta}^B, [\![\lambda']\!]_{\Delta}^B$ and reverse samples Δ_A and $[\![\lambda']\!]_{\Delta}^A, [\![\lambda']\!]_{\Delta}^A$ where $\Delta := \Delta_A || \Delta_B || 1$.

Garble: $\mathcal{S}_B$ does nothing.

Encode Inputs: $\mathcal{S}_B$ simulates **Open** and **Fix** to obtain masked inputs and labels $\{\Lambda'_w, L'_{w, \Lambda'_w}\}_{w \in \mathcal{I}}$ using input $\mathbf{x}' = 0$ and extracts $\mathcal{A}$'s input $\mathbf{y}'$. $\mathcal{S}_B$ defines errors on input wires as $e_w = y_w \oplus y'_w$ for $w \in \mathcal{I}_B$ and $e_w = 0$ for $w \in \mathcal{I}_A$.

Evaluate: $\mathcal{S}_B$ receives $\mathcal{GC}'_B$ from $\mathcal{A}$. $\mathcal{S}_B$ extracts the errors e_w introduced on output wires of non-free gates as follows: It first evaluates the received $\mathcal{GC}'_B$ with received input masks and labels to obtain masked values. It then recomputes $\mathcal{GC}'_B$ using an honest $\mathcal{A}$'s approach (using $\mathcal{A}$'s internal randomness) and evaluates this to obtain expected masked values. It sets $e_w = 1$ if the obtained masked values do not equal the expected masked values on wire w . It also extracts $[\![z'_w]\!]_{\Delta_B}^B$ for $w \in \mathcal{I} \cup \mathcal{W} \cup \mathcal{O}$.

Consistency Check If $e_w \neq 0$ for any $w \in \mathcal{I} \cup \mathcal{W}$, then $\mathcal{S}_B$ samples $h_A \leftarrow \mathbb{F}_{2^\kappa}$ uniformly at random. Otherwise, $\mathcal{S}_B$ computes $h_A = H(\{V_w^B\}_{w \in \mathcal{I} \cup \mathcal{W}})$ where $V_w^B = [\![z_w]\!]_{\Delta_A}^B \oplus [\![z'_w]\!]_{\Delta_B}^B \oplus [\![\lambda_w]\!]_{\Delta_B}^B \oplus [\![\lambda'_w]\!]_{\Delta_A}^B \oplus (z_w \oplus \lambda_w) \Delta_B$ as per the protocol specification. $\mathcal{S}_B$ sends h_A to $\mathcal{A}$. Note that $\mathcal{S}_B$ uses $\mathcal{P}_B$'s strategy for computing the hash, not $\mathcal{P}_A$'s strategy. This is because $\mathcal{P}_A$'s garbling is now simulated so those values will be inconsistent, and it is appropriate to use $\mathcal{P}_B$'s values if $e_w = 0$, as $V_w^A = V_w^B$ (Lemma 9).

Decode Outputs If the previous step does not abort, then $\mathcal{S}_B$ receives the correct output $\mathbf{z} = \mathcal{C}(\mathbf{x}, \mathbf{y})$ from $\mathcal{F}_{2\text{pc}}$. Using masked values Λ_w obtained from the simulated garbling, $\mathcal{S}_B$ simulates **Open** to reveal $\Lambda_w \oplus z_w$ as the output mask so that an honest $\mathcal{A}$ obtains output $\mathbf{z}$ as required.

Now consider the series of hybrids where the first one is the real protocol execution and the last one is the above simulated execution.

Hybrid 1. This is the hybrid execution where $\mathcal{S}_B$ simulates $\mathcal{F}_{\text{cpre}}$ and follows the $\Pi_{2\text{pc-tensor}}$ protocol by playing the role of an honest $\mathcal{P}_A$, using the actual input $\mathbf{x}$.

Hybrid 2. In this hybrid, we change how $\mathcal{S}_B$ computes the value h_A in the consistency check, as described above. In particular, it first extracts inputs $\mathbf{y}$ and $\mathbf{y}'$ and computes errors e_w from the received $\mathcal{GC}'_B$ as discussed in the simulator's *Encode Inputs* and *Evaluate* in the second execution. It also computes $[\![z_w]\!]_{\Delta_A}^B, [\![z'_w]\!]_{\Delta_B}^B$ during *Evaluate* of both executions like the simulator. Finally, it computes h_A as described in the simulator's consistency check.

By Lemma 9, we have $V_w^A = V_w^B \oplus e_w(\Delta_A \oplus \Delta_B)$. Therefore, if $e_w = 0$ for all wires then the computation of h_A is unchanged. If there exists some $e_w \neq 0$ then h_A appears uniformly random to $\mathcal{A}$, except it if queries Δ_A which happens with probability at most $\tau \cdot 2^{-\kappa}$ since Δ_A is independent of its view. Therefore, this hybrid is $(\tau \cdot 2^{-\kappa})$ -indistinguishable from the previous one.

Hybrid 3. In this hybrid, we modify how $\mathcal{S}_B$ garbles $\mathcal{GC}_A$, encodes inputs in the first execution and decodes outputs, as described above.

$\mathcal{GC}_A$ and $\{L_{w, \Lambda_w}\}_{w \in \mathcal{I}}$ received by $\mathcal{A}$ are, in isolation, ϵ_{ccrnd} indistinguishable from the previous hybrid by security of $\mathcal{S}_{\text{obv}}$. We show how they remain indistinguishable in the context of the protocol, as this effectively changes $\mathcal{S}_B$'s input to $\mathbf{x} = \mathbf{0}$. The previous hybrid modifies the consistency check so that it does not use values obtained via $\mathcal{S}_B$'s garbling so the probability of abort is unaffected. Further, $\mathcal{S}_B$ now decodes outputs such that it reveals $\mathbf{z} = \mathcal{C}(\mathbf{x}, \mathbf{y})$ received from $\mathcal{F}_{2\text{pc}}$ to an honest $\mathcal{A}$. Therefore, this is ϵ_{ccrnd} indistinguishable from the previous hybrid by the circular correlation robust under naturally derived keys property of $\mathcal{H}_{\text{ccrnd}}$.

Hybrid 4. In this hybrid we change $\mathcal{S}_B$'s input in the second execution from $\mathbf{x}$ to $\mathbf{0}$. The masked inputs that $\mathcal{A}$ receives in the second execution are distributed in the same way because masks are picked uniformly at random. We have defined e_w exactly such that the probability there exists $e_w \neq 0$ for $w \in \mathcal{W}$ changes with probability at most $2^{-\rho}$ due to selective-failure resistance of $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ (Lemma 8). Therefore, this hybrid is $(2^{-\rho})$ -indistinguishable from the previous one. This is exactly the ideal distribution.

Altogether, the ideal world and real world executions are $(\epsilon_{\text{ccrnd}} + \tau \cdot 2^{-\kappa} + 2^{-\rho})$ -indistinguishable in the corrupted $\mathbf{P}_B$ case. A similar proof for the case of corrupted $\mathbf{P}_A$ implies that the protocol $\Pi_{2\text{pc-tensor}}$ securely realizes $\mathcal{F}_{2\text{pc}}$ in the $\mathcal{F}_{\text{cpre}}$ -hybrid model. □

B Applications

In this section, we recap some applications of tensor gates mentioned in [HK21b], namely using our “short” input tensor gates to compute generalized tensor products and applying that to matrix and integer multiplication.

B.1 Generalized Outer Product

We first show how to efficiently compute the tensor product of two binary vectors of arbitrary length. Tensor gates in our distributed garbling are limited to inputs of length at most logarithmic in the circuit input size due to their exponential scaling in computation.

To support longer vectors, [HK21b] introduces a tiling approach which we modify to make black-box calls to short-input tensor gates: the masked input $\mathbf{x} \oplus \lambda_{\mathbf{x}}$ (resp. $\mathbf{y} \oplus \lambda_{\mathbf{y}}$) is divided into $N := \lceil n/t \rceil$ (resp. $M := \lceil m/t \rceil$) chunks of dimension $t \times t$. We use short tensor gates, setting $t = c \log \kappa$ for some $c > 0$, to compute the NM many pairwise tensor products of chunks. The full matrix is reconstructed by concatenating these submatrices.

Each tensor product of pairwise chunks has amortized communication cost $O(\log \kappa)$ ciphertexts so the tiling approach reduces the communication cost from $O(nm)$ ciphertexts to $O(NM \log \kappa) = O(nm / \log \kappa)$ ciphertexts, yielding a $c \log \kappa$ factor improvement.

The number of hashes computed for a $t \times t$ tensor product is $2^t \cdot t = O(\kappa^c \log \kappa)$. This approach computes NM such tensor products leading to a total of $O(NM \kappa^c \log \kappa) = O(nm \kappa^c / \log \kappa)$ hashes compared to $O(nm)$ hashes using the naive approach. Therefore, the total computation increases by a factor of $\kappa^c / \log \kappa$. However, since communication is typically the bottleneck in GC protocols, this leads to improvement in run time for well-selected chunking parameters (see Appendix C).

B.2 Binary Matrix Multiplication

Matrix products can be computed efficiently by summing tensor products of columns and rows. Given binary input matrices $\mathbf{X} \in \mathbb{F}_2^{n \times m}$ and $\mathbf{Y} \in \mathbb{F}_2^{m \times \ell}$, the product $\mathbf{C} = \mathbf{X} \cdot \mathbf{Y}$ can be expressed as a sum of m outer products: for each $i \in [m]$, compute the outer product of column i of $\mathbf{X}$ and row i of $\mathbf{Y}$, and XOR the resulting matrices together.

Our protocol for generalized tensor products, like that of [HK21b], can be applied directly as a black box to each pair of vectors. The reduction in cost per tensor product (via the tiling approach) reduces the overall communication from $O(nm\ell)$ to $O(nm\ell / \log \kappa)$ ciphertexts.

B.3 Integer Multiplication

Tensor gates also enables more efficient garbling of integer schoolbook multiplication. In the schoolbook method, the product $\mathbf{x} \cdot \mathbf{y}$ is computed as a sum of terms of the form $x_i \cdot \mathbf{y} \ll i$, where $i \in [n]$ and $\mathbf{y} \ll i$ denotes $\mathbf{y}$ left-shifted by i bits. This corresponds to computing all pairwise products $x_i \cdot y_j$ for $i, j \in [n]$, which are precisely the entries of the binary tensor product $\mathbf{x} \otimes \mathbf{y}$. Using the tiling technique and tensor gates, these pairwise products can be computed efficiently in an authenticated garbled circuit.

While the computation of $\mathbf{x} \otimes \mathbf{y}$ is improved, the summation of the resulting partial products is still performed inside GC using traditional methods. This step becomes the new bottleneck of performance. This optimized multiplication routine can also serve as a base case in recursive schemes like Karatsuba multiplication.

C Experiments

In this section, we benchmark the online phase of $\Pi_{2\text{pc-tensor}}$ for computing tensor products of boolean vectors. We use the generalized tensor product technique (Appendix B) to handle arbitrary length inputs. We record our methodology and present the results of our findings.

C.1 Methodology

C.1.1 Implementation Details

We implemented the online phase of $\Pi_{2\text{pc-tensor}}$ in Rust and have made the source code publicly available.⁴ Our implementation uses an idealized preprocessing functionality to generate the correlations required by our distributed tensor garbling scheme ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$), as we only benchmark the online phase. The preprocessing phase’s cost is identical to the state of the art authenticated garbling protocol [CWYY23] that would handle tensor gates by decomposing them into AND gates, so we do not implement it. ExtBit is instantiated as the least significant bit of each label (IT-MAC component). Labels are 128 bits long; the security parameter is thus set to $\kappa = 127$. We use the Criterion Rust crate [AH25] to benchmark run time. The benchmark proceeds as follows: first, the circuit is garbled, then ciphertexts are sent, and finally the garbled circuit evaluated. Note that for dual execution, both parties can first simultaneously garble their circuits, then transfer them, and then simultaneously evaluate. We did not parallelize the tensor product for the purposes of our experiments, but expect this to make our contribution even greater as the one-hot garbling technique lends itself to efficient parallelization.

C.1.2 Computation Setup

We ran all experiments on a MacBook Pro laptop with an M4 Pro 2.6 GHz processor and 24 GB of RAM. We used the Criterion crate to run and to normalize tests with a 3 second warm-up per test, with each test taken at least 100 times. Garbling, transmission, and evaluation are ran sequentially on the same machine.

C.1.3 Communication Setup

A 100 Mbps connection was simulated for communication between the garbler and the evaluator. No jitter or delay was simulated, as all ciphertexts are sent in only one transmission, and interaction in one round of the online phase of dual execution is one-way. Wall clock time and the size of ciphertexts in bytes are recorded.

C.1.4 Experiments

We simulated a full round of garbling, transmission, and evaluations for $N \times N$ tensor products where $N \in \{8, 16, 32, 64, 128, 256\}$ with tile sizes ranging from 1 to 8. As our distributed tensor garbling scheme ($\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}}$) reduces to distributed half gates ($\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}}$) when the tile size is set to 1, we compare all benchmarks against those with a tile size of 1. In this way, we test the efficiency of our protocol in computing a binary tensor product against a naively implemented circuit that computes a tensor product using N^2 AND gates.

C.2 Results

In accordance with our analysis in Appendix B, an increase in the tile size increases the number of hashes computed but reduces the amount of garbled material communicated. As a result, we see that the wall clock time decreases until it reaches a minimum and then increases for larger tile sizes. We found that across all

	Standard	Ours	Improvement
Comm. (KB)	524.3	96.9	$5.41\times$
Time (ms)	47.9	14.3	$3.35\times$

experiments, a tile size of 6 was optimal in wall clock time. For a tensor product of two 128-bit vectors, with a tile size of 6, we observe the following advantage from our scheme in terms of time and communication:

Figure 1 shows our results for $N \times N$ tensor products for $N = 64, 128, 256$ across tile sizes ranging from 1 through 8. As can be observed from the plots, the baseline the incurs the most cost in terms of time and communication. We see advantages in time and communication until the optimal tile size of 6, after which communication continues decreasing at the cost of time incurred by computation.

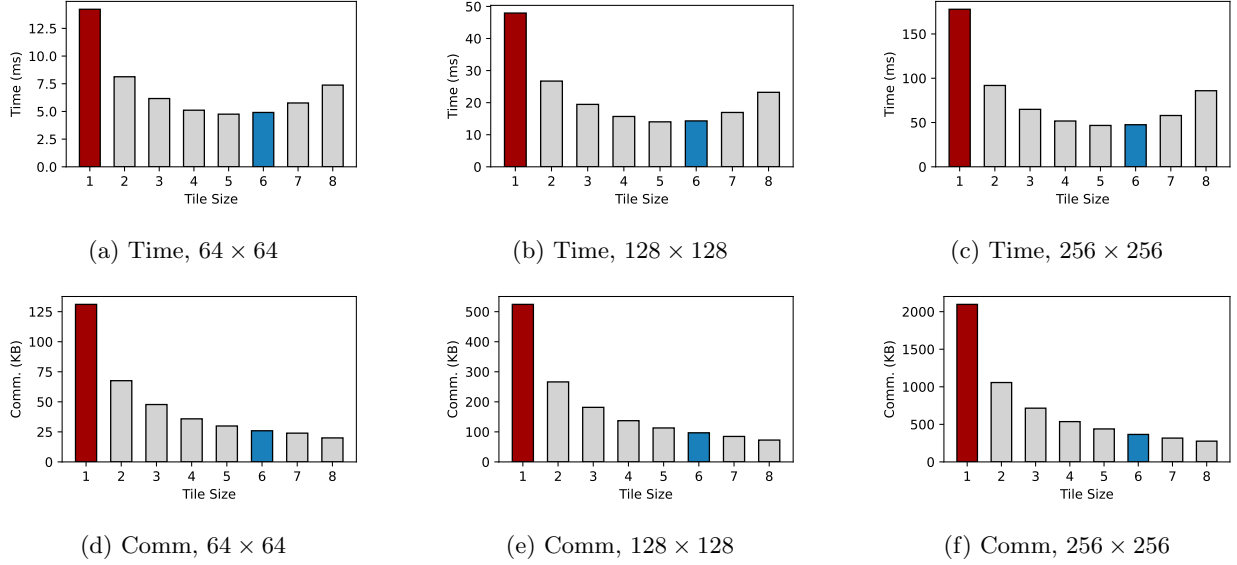


Figure 1: Wall clock time (ms) and communication (KB) for tile sizes 1 through 8 for various input lengths, with the baseline (tile size of 1) highlighted in red, and the optimal (tile size 6) highlighted in blue.

⁴The repository can be found at: <https://github.com/Authenticated-Tensor-Garbling>

D Equivalence of Distributed Half-Gates

In this section, we explicitly spell out the ciphertexts sent by G to E and how output labels are set in our modified distributed half gates scheme ($\text{Garble}_{\text{HG}}, \text{Eval}_{\text{HG}}$) to justify its equivalence to the one presented in [KRRW18]. First note when we say parties compute GarbleAnd on garbled shares $\llbracket x \rrbracket := (X, X \oplus x\Delta)$ and $\llbracket y \rrbracket := (Y, Y \oplus y\Delta)$, we mean that garbler outputs $C = H(X)$ and sends $G = H(X) \oplus H(X \oplus \Delta) \oplus Y$ to evaluator who outputs $H(X \oplus x\Delta) \oplus x(G \oplus Y \oplus y\Delta) = C \oplus xy\Delta$. This constitutes a garbled share $\llbracket xy \rrbracket = (C, C \oplus xy\Delta)$ as required and was initially presented in [ZRE15].

A subtle difference between the notation we use for authenticated garbling and prior protocols is that we use labels to share *semantic* values (as done in semi-honest garbling protocols) rather than *masked values* (done in [WRK17, KRRW18, DILO22, CWYY23]). This does not affect security or efficiency in any way and we find it helpful to use notation that is consistent with the semi-honest setting. However, for the sake of illustrate the equivalence of our presentation of distributed half-gates (Section 3.1.2) and [KRRW18], we adopt their notation where parties hold garbled shares $\llbracket x \oplus \lambda_x \rrbracket, \llbracket y \oplus \lambda_y \rrbracket$ and wish to obtain $\llbracket z \oplus \lambda_z \rrbracket$.

Equivalence of Distributed Half Gates

Inputs: Parties preprocess shares $\llbracket \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket, \llbracket \lambda_x \lambda_y \rrbracket, \llbracket \lambda_z \rrbracket$. E holds masked values $x \oplus \lambda_x, y \oplus \lambda_y$. Parties hold shares $\llbracket x \oplus \lambda_x \rrbracket := (\mathbb{L}_{x,0}, \mathbb{L}_{x,\Lambda_x})$ and $\llbracket y \oplus \lambda_y \rrbracket := (\mathbb{L}_{y,0}, \mathbb{L}_{y,\Lambda_y})$.

Procedure:

1. Parties locally compute $\llbracket x \rrbracket = \llbracket x \oplus \lambda_x \rrbracket \oplus \llbracket \lambda_x \rrbracket, \llbracket y \rrbracket = \llbracket y \oplus \lambda_y \rrbracket \oplus \llbracket \lambda_y \rrbracket$.
2. Parties compute

$$\begin{aligned} \llbracket (x \oplus \lambda_x) \lambda_y \rrbracket &= \text{GarbleAnd}(\llbracket x \oplus \lambda_x \rrbracket, \llbracket \lambda_y \rrbracket), \\ \mathbb{L}_{z,0}^{(0)} &:= H(\mathbb{L}_{x,0}), \quad G_z^{(0)} := H(\mathbb{L}_{x,0}) \oplus H(\mathbb{L}_{x,1}) \oplus \llbracket \lambda_y \rrbracket^G \\ \mathbb{L}_{z,\Lambda_z}^{(0)} &:= H(\mathbb{L}_{x,\Lambda_x}) \oplus \Lambda_x(G_z^{(0)} \oplus \llbracket \lambda_y \rrbracket^E) \\ \llbracket (y \oplus \lambda_y) x \rrbracket &= \text{GarbleAnd}(\llbracket y \oplus \lambda_y \rrbracket, \llbracket x \rrbracket) \\ \mathbb{L}_{z,0}^{(1)} &:= H(\mathbb{L}_{y,0}), \quad G_z^{(1)} := H(\mathbb{L}_{y,0}) \oplus H(\mathbb{L}_{y,1}) \oplus \mathbb{L}_{x,0} \oplus \llbracket \lambda_x \rrbracket^G \\ \mathbb{L}_{z,\Lambda_z}^{(1)} &:= H(\mathbb{L}_{y,\Lambda_y}) \oplus \Lambda_y(G_z^{(1)} \oplus \llbracket \lambda_x \rrbracket^E \oplus \mathbb{L}_{x,\Lambda_x}) \end{aligned}$$

using 2κ bits of communication.

3. Parties locally compute

$$\begin{aligned} \llbracket xy \oplus \lambda_z \rrbracket &= \llbracket (x \oplus \lambda_x) \lambda_y \rrbracket \oplus \llbracket (y \oplus \lambda_y) x \rrbracket \oplus \llbracket \lambda_x \lambda_y \oplus \lambda_z \rrbracket. \\ \mathbb{L}_{z,0} &:= \mathbb{L}_{z,0}^{(0)} \oplus \mathbb{L}_{z,0}^{(1)} \oplus \llbracket \lambda_z^* \rrbracket^G \oplus \llbracket \lambda_z \rrbracket^G \\ \mathbb{L}_{z,\Lambda_z} &:= \mathbb{L}_{z,\Lambda_z}^{(0)} \oplus \mathbb{L}_{z,\Lambda_z}^{(1)} \oplus \llbracket \lambda_z^* \rrbracket^E \oplus \llbracket \lambda_z \rrbracket^E \end{aligned}$$

G sends $\llbracket xy \rrbracket \oplus \llbracket \lambda_z \rrbracket$ to E to reveal $xy \oplus \lambda_z$ (Remark 2).

Outputs: Parties hold garbled shares $\llbracket xy \oplus \lambda_z \rrbracket$, E holds masked value $xy \oplus \lambda_z$.

Putting everything together, an AND gate is computed by G setting

$$\mathbb{L}_{z,0} = H(\mathbb{L}_{x,0}) \oplus H(\mathbb{L}_{y,0}) \oplus \llbracket \lambda_z^* \rrbracket^G \oplus \llbracket \lambda_z \rrbracket^G,$$

sending

$$\begin{aligned} G_z^{(0)} &:= H(\mathsf{L}_{x,0}) \oplus H(\mathsf{L}_{x,1}) \oplus \llbracket \lambda_y \rrbracket^{\mathsf{G}}, \\ G_z^{(1)} &:= H(\mathsf{L}_{y,0}) \oplus H(\mathsf{L}_{y,1}) \oplus \mathsf{L}_{x,0} \oplus \llbracket \lambda_x \rrbracket^{\mathsf{G}}, \end{aligned}$$

and E computing

$$\mathsf{L}_{z,\Lambda_z} = H(\mathsf{L}_{x,\Lambda_x}) \oplus H(\mathsf{L}_{y,\Lambda_y}) \oplus \Lambda_x(G_z^{(0)} \oplus \llbracket \lambda_y \rrbracket^{\mathsf{G}}) \oplus \Lambda_y(G_z^{(1)} \oplus \mathsf{L}_{x,\Lambda_x} \oplus \llbracket \lambda_x \rrbracket^{\mathsf{G}}).$$

This is precisely the formulation of distributed half-gates provided in [KRRW18], with our notation for IT-MACs from Section 2.4.

E Compressed Preprocessing [CWYY23]

For completeness, we present the compressed preprocessing functionality $\mathcal{F}_{\text{cpre}}$ [CWYY23]. We introduce a compression parameter σ as input to the functionality which controls the entropy of the preprocessed wire masks. Originally, this was set to $\sigma = 2\rho$ [CWYY23] but our distributed tensor garbling scheme requires $\sigma = O(\rho \log \kappa)$ for $2^{-\rho}$ selective failure resistance (Lemma 8). We instantiate this functionality using Π_{cpre} [CWYY23] (modifying the compression parameter σ and therefore the length of compressed mask vector L) and refer the reader to the original paper for a proof of security.

Theorem 3. *Protocol Π_{cpre} [CWYY23] securely realizes $\mathcal{F}_{\text{cpre}}$ against malicious adversaries in the $(\mathcal{F}_{\text{COT}}, \mathcal{F}_{\text{bCOT}}, \mathcal{F}_{\text{DVZK}}, \mathcal{F}_{\text{EQ}}, \mathcal{F}_{\text{Rand}})$ -hybrid model.*

This protocol generates AND triples $(\lambda_x, \lambda_y, \lambda_x \lambda_y)$ ⁵ but $(\text{Garble}_{\text{Tensor}}, \text{Eval}_{\text{Tensor}})$ requires tensor triples $(\lambda_x, \lambda_y, \lambda_x \otimes \lambda_y)$. Since a tensor triple is simply a combination of structured AND triples $(\lambda_x \otimes \lambda_y)_{ij} = (\lambda_x)_i (\lambda_y)_j$, it is straightforward to generate one tensor triple using nm AND triples where $n = |\lambda_x|, m = |\lambda_y|$. We do not try to further optimize preprocessing as the communication cost is independent of κ and does not affect the asymptotic complexity of our protocol. In particular, our preprocessing protocol has a one-way amortized communication of $2nm$ bits per tensor gate (2 bits per AND gate [CWYY23]).

We leave it for future work to reduce the amortized communication complexity of preprocessing dual authenticated tensor triples for a $(\otimes, m, n)$ gate from $O(mn)$ bits to $O(m + n)$ bits, but note that even with this our authenticated garbling protocol in Section 5 would have $O(n + m)\kappa + O(nm)$ amortized communication complexity because of the online phase.

E.1 Communication Complexity

For $\sigma = O(\rho \log \kappa)$, the length L of the compressed mask vector is still logarithmic in the number of AND gates so we have the exact same amortized communication cost of 2 bits per AND triple as [CWYY23].

Ideal Functionality $\mathcal{F}_{\text{cpre}}$ - Compressed Preprocessing

This functionality is parameterized by a Boolean circuit $\mathcal{C}$ consisting of a list of gates in the form of $(\alpha, \beta, \gamma, T)$ where $T \in \{\oplus, \wedge\}$ and a compression parameter σ . Let $n = |\mathcal{W}|$ be the number of output wires of non-free gates. Set $L = \lceil \rho + \sigma \cdot \log(\frac{en}{\sigma}) + \frac{\log \sigma}{2} \rceil$ be the length of the compressed mask vector. It runs with parties $\mathcal{P}_A, \mathcal{P}_B$ and the ideal-world adversary $\mathcal{S}$, and operates as follows:

Initialize. Sample two global keys $\Delta_A, \Delta_B \in \mathbb{F}_{2^\kappa}$ as follows:

- If $\mathcal{P}_A$ is honest, sample $\Delta_A \leftarrow \mathbb{F}_{2^\kappa}$ such that $\text{lsb}(\Delta_A) = 1$. Otherwise, receive $\Delta_A \in \mathbb{F}_{2^\kappa}$ with $\text{lsb}(\Delta_A) = 1$ from $\mathcal{S}$.
- If $\mathcal{P}_B$ is honest, sample $\Delta_B \leftarrow \mathbb{F}_{2^\kappa}$ such that $\text{lsb}(\Delta_A \Delta_B) = 1$ and $\text{msb}(\Delta_B) = 1$. Otherwise, receive $\Delta_B \in \mathbb{F}_{2^\kappa}$ with $\text{msb}(\Delta_B) = 1$ from $\mathcal{S}$, and then re-sample $\Delta_A \leftarrow \mathbb{F}_{2^\kappa}$ such that $\text{lsb}(\Delta_A \Delta_B) = 1$ and $\text{lsb}(\Delta_A) = 1$.
- Store (Δ_A, Δ_B) , and output Δ_A and Δ_B to $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively.

Macro. $\text{Auth}_A(\mathbf{x}, \ell)$ (this is an internal subroutine only)

- If $\mathcal{P}_B$ is honest, sample $\mathbf{K}_B[\mathbf{x}] \leftarrow \mathbb{F}_{2^\kappa}^\ell$; otherwise, receive $\mathbf{K}_B[\mathbf{x}] \in \mathbb{F}_{2^\kappa}^\ell$ from $\mathcal{S}$.
- If $\mathcal{P}_A$ is honest, compute $\mathbf{M}_A[\mathbf{x}] := \mathbf{K}_B[\mathbf{x}] + \mathbf{x} \cdot \Delta_B \in \mathbb{F}_{2^\kappa}^\ell$. Otherwise, receive $\mathbf{M}_A[\mathbf{x}] \in \mathbb{F}_{2^\kappa}^\ell$ from $\mathcal{S}$, and recompute $\mathbf{K}_B[\mathbf{x}] := \mathbf{M}_A[\mathbf{x}] + \mathbf{x} \cdot \Delta_B \in \mathbb{F}_{2^\kappa}^\ell$.
- Send $(\mathbf{x}, \mathbf{M}_A[\mathbf{x}])$ to $\mathcal{P}_A$ and $\mathbf{K}_B[\mathbf{x}]$ to $\mathcal{P}_B$.

$\text{Auth}_B(\mathbf{x}, \ell)$ can be defined similarly by swapping the roles of $\mathcal{P}_A$ and $\mathcal{P}_B$.

Preprocess the circuit with compressed wire masks. Sample $\mathbf{M} \leftarrow \mathbb{F}_2^{n \times L}$, and then execute as follows:

⁵Technically, $\mathcal{F}_{\text{cpre}}$ outputs $\llbracket \mathbf{a} \rrbracket_{\Delta_B}, \llbracket \mathbf{b} \rrbracket_{\Delta_A}$ but this can be locally converted to $\llbracket \boldsymbol{\lambda} \rrbracket_{\Delta_A \parallel \Delta_B \parallel 1}$ where $\boldsymbol{\lambda} := \mathbf{a} \oplus \mathbf{b}$ (see Section 2.4).

- For $w \in \mathcal{I}_A$, set $b_w = 0$ and define $\llbracket b_w \rrbracket_{\Delta_A}$.
 - If P_A is honest, sample $\mathbf{a} \leftarrow \mathbb{F}_2^m$; otherwise, receive $\mathbf{a} \in \mathbb{F}_2^m$ from $\mathcal{S}$. Then, execute $\text{Auth}_A(\mathbf{a}, m)$ to generate $\llbracket \mathbf{a} \rrbracket_{\Delta_B}$. For each wire $w \in \mathcal{I}_A \cup \mathcal{I}_B \cup \mathcal{W}$, define a_w as the wire mask held by P_A .
 - If P_B is honest, sample $\mathbf{b}^* \leftarrow \mathbb{F}_2^L$; otherwise, receive $\mathbf{b}^* \in \mathbb{F}_2^L$ from $\mathcal{S}$. Let $\mathbf{b} = \mathbf{M} \cdot \mathbf{b}^*$. Run $\text{Auth}_B(\mathbf{b}, n)$ to generate $\llbracket \mathbf{b} \rrbracket_{\Delta_A}$. For each wire $w \in \mathcal{I}_B \cup \mathcal{W}$, define b_w as the wire mask held by P_B .
 - In a topological order, for each gate $(\alpha, \beta, \gamma, T)$, do the following:
 - If $T = \oplus$, compute $\llbracket a_\gamma \rrbracket_{\Delta_B} := \llbracket a_\alpha \rrbracket_{\Delta_B} \oplus \llbracket a_\beta \rrbracket_{\Delta_B}$ and $\llbracket b_\gamma \rrbracket_{\Delta_A} := \llbracket b_\alpha \rrbracket_{\Delta_A} \oplus \llbracket b_\beta \rrbracket_{\Delta_A}$.
 - If $T = \otimes$, execute as follows:
 1. If P_A is honest, then sample $\hat{a}_\gamma \leftarrow \mathbb{F}_2$, else receive $\hat{a}_\gamma \in \mathbb{F}_2$ from $\mathcal{S}$.
 2. If P_B is honest, then compute $\hat{b}_\gamma := (a_\alpha \oplus b_\alpha) \wedge (a_\beta \oplus b_\beta) \oplus \hat{a}_\gamma$ for $i \in [n], j \in [m]$. Otherwise, receive $\hat{b}_\gamma \in \mathbb{F}_2$ from $\mathcal{S}$, and re-compute $\hat{a}_\gamma := (a_\alpha \oplus b_\alpha) \wedge (a_\beta \oplus b_\beta) \oplus \hat{b}_\gamma$ for $i \in [n], j \in [m]$.
- Let $\hat{\mathbf{a}}$ and $\hat{\mathbf{b}}$ be the vectors consisting of $\hat{a}_w$ and $\hat{b}_w$ for $w \in \mathcal{W}$. Run $\text{Auth}_A(\hat{\mathbf{a}})$ and $\text{Auth}_B(\hat{\mathbf{b}})$ to generate $\llbracket \hat{\mathbf{a}} \rrbracket_{\Delta_B}$ and $\llbracket \hat{\mathbf{b}} \rrbracket_{\Delta_A}$, respectively.
- Output M and $(\llbracket \mathbf{a} \rrbracket_{\Delta_B}, \llbracket \hat{\mathbf{a}} \rrbracket_{\Delta_B}, \llbracket \mathbf{b}^* \rrbracket_{\Delta_A}, \llbracket \hat{\mathbf{b}} \rrbracket_{\Delta_A})$ to P_A and P_B .